Telecommunication Systems (2026) 89:42
https://doi.org/10.1007/s11235-026-01426-y

Deep learning approaches for computation offloading in edge computing: A critical review

Sapthagiri Miriyala[1] · **Venkata Ramireddy Chirra**[1]

Received: 21 June 2025 / Accepted: 11 February 2026 / Published online: 24 February 2026

Abstract

The paradigm shift from centralized cloud computing to Multi-Access Edge Computing (MEC) has been necessitated by the exponential growth of the Internet of Things (IoT) and the stringent latency requirements of 5G/6G applications. However, the inherent dynamism of edge environments, characterized by fluctuating channel conditions, heterogeneous device capabilities, and finite energy budgets, renders traditional static optimization techniques insufficient. This survey provides a critical review of Deep Learning (DL) and Deep Reinforcement Learning (DRL) as pivotal enablers for autonomous computation offloading. This work systematically dissects the algorithmic trade-offs between Value-Based (e.g., DQN) and Policy-Gradient (e.g., PPO, SAC, A3C) architectures, evaluating their suitability for continuous control, multi-agent coordination, and resource-constrained inference. We further bridge the gap between theoretical algorithms and practical telecommunications deployment by mapping DRL strategies to European Telecommunications Standards Institute (ETSI) MEC specifications and Open Radio Access Network (O-RAN) architectural standards. The review analyzes the "optimization trade-offs" inherent in offloading, specifically the conflict between service immediacy (latency) and resource conservation (energy/security). Finally, we synthesize key challenges hindering large-scale adoption, including the need for lightweight "extreme edge" models, robustness against non-stationary environments, and privacy-preserving federated learning, thereby providing a rigorous roadmap for the evolution of self-optimizing, intelligent edge ecosystems.

Keywords Multi-Access Edge Computing · Deep Reinforcement Learning · Computation Offloading · Resource Allocation · Federated Learning · IoT

Nomenclature

A2C	Advantage Actor-Critic
A3C	Asynchronous Advantage Actor-Critic
AI	Artificial Intelligence
BS	Base Station
CC	Cloud Computing
CNN	Convolutional Neural Network
CSI	Channel State Information
CTDE	Centralized Training Decentralized Execution
DAG	Directed Acyclic Graph
DDPG	Deep Deterministic Policy Gradient
DDQN	Double Deep Q-Network
DL	Deep Learning
DNN	Deep Neural Network
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ETSI	European Telecommunications Standards Institute
FDRL	Federated Deep Reinforcement Learning
FL	Federated Learning
GNN	Graph Neural Network
GPU	Graphics Processing Unit
IID	Independent and Identically Distributed
IIoT	Industrial Internet of Things
IoT	Internet of Things
IoUAV	Internet of Unmanned Aerial Vehicles
IoV	Internet of Vehicles
LLM	Large Language Model
LSTM	Long Short-Term Memory
MADDPG	Multi-Agent Deep Deterministic Policy Gradient
MADRL	Multi-Agent Deep Reinforcement Learning
MAML	Model-Agnostic Meta-Learning
MAPPO	Multi-Agent Proximal Policy Optimization
MARL	Multi-Agent Reinforcement Learning
MCC	Mobile Cloud Computing

✉ Venkata Ramireddy Chirra
venkataramireddy.chirra@vitap.ac.in

[1] VIT-AP University, Amaravati, India

MDP	Markov Decision Process
MEC	Multi-Access Edge Computing
MINLP	Mixed Integer Non-Linear Programming
ML	Machine Learning
mMTC	Massive Machine-Type Communication
NAS	Neural Architecture Search
NOMA	Non-Orthogonal Multiple Access
O-RAN	Open Radio Access Network
OD-SARSA	On-Policy Deep SARSA
PLS	Physical Layer Security
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimization
PSO	Particle Swarm Optimization
QoS	Quality of Service
RAN	Radio Access Network
RIC	RAN Intelligent Controller
RL	Reinforcement Learning
RNIS	Radio Network Information Service
RNN	Recurrent Neural Network
RSU	Roadside Unit
SAC	Soft Actor-Critic
SARSA	State-Action-Reward-State-Action
SCA	Successive Convex Approximation
SDN	Software-Defined Networking
TD3	Twin Delayed Deep Deterministic Policy Gradient
UAV	Unmanned Aerial Vehicle
UE	User Equipment
URLLC	Ultra-Reliable Low-Latency Communication
V2V	Vehicle-to-Vehicle
V2X	Vehicle-to-Everything
VEC	Vehicular Edge Computing

1 Introduction

The rapid adoption of internet-enabled mobile technologies is transforming the global digital landscape. By 2025, the number of devices connected to the internet is anticipated to reach 80 billion, generating an estimated 180 trillion gigabytes of global data [1]. This proliferation is driving the emergence of next-generation applications such as autonomous vehicles, augmented reality, real-time video analytics, and industrial automation that require processing with stringent latency bounds often measured in single-digit milliseconds [2]. The expansion of the Internet of Things (IoT) is not only reshaping technological landscapes but also driving significant economic impact, with intelligent healthcare applications alone projected to contribute up to $2.5 trillion in annual revenue by 2025. However, the staggering volume of data generated by these ubiquitous devices, combined with the acceleration of 5G technology, presents substantial computational challenges that often exceed the processing capabilities of standard user equipment [3].

In light of these advancements, conventional cloud-based architectures face inherent physical limitations. While cloud computing offers virtually unlimited scalable resources, it struggles to satisfy the requirements of delay-sensitive applications due to network propagation delays, congestion, and the unpredictability of wide-area network performance [4]. To address these bottlenecks, researchers have established a "computational continuum" that decentralizes data processing across edge nodes and cloud infrastructure [5]. Multi-Access Edge Computing (MEC), as standardized by the European Telecommunications Standards Institute (ETSI) [6], has emerged as the architectural response to these challenges. By situating computational resources co-located with base stations, MEC fundamentally alters the optimization landscape: whereas cloud computing optimizes for scale, MEC centers on minimizing latency while managing severely constrained resources, including finite energy budgets and restricted memory footprints [7, 8].

A core advantage of this architecture is *computation offloading*:the ability to migrate computationally demanding workloads from resource-constrained mobile devices to more powerful edge servers. Delegating tasks to edge nodes significantly improves device battery life and enhances application performance [9, 10]. However, the computation offloading decision represents a complex optimization problem. For any given task, the system must determine whether execution should proceed locally, be offloaded to an edge server, or be forwarded to the cloud. This decision is governed by numerous dynamically varying factors, including channel signal-to-noise ratios, server computational load, task complexity, and user mobility [11, 12].

The state space for these decisions grows combinatorially with the number of devices and potential offloading targets, rendering exact optimization approaches computationally intractable for realistic, real-time deployments. Consequently, Deep Reinforcement Learning (DRL) has emerged as a compelling solution for this complexity. By combining the representational power of deep neural networks with the sequential decision-making framework of reinforcement learning, DRL agents can learn effective offloading policies through interaction with the environment without requiring explicit models of system dynamics [13]. This survey explores how DRL is revolutionizing MEC, addressing the critical challenges of dynamic resource allocation in the era of hyper-connected smart computing.

1.1 Motivation and related surveys

The integration of DL within MEC has emerged as a transformative strategy for optimizing computational workloads. While standard edge devices often lack the capacity to han-

dle the substantial computational resources required by Deep Neural Networks (DNNs), leveraging DL-based optimization within MEC enables adaptive, intelligent task offloading. This synergy is crucial for the evolution of next-generation smart computing frameworks, moving beyond static rule-based systems toward dynamic, self-learning architectures.

Although numerous surveys have examined computation offloading, a critical analysis reveals distinct evolution patterns and persisting gaps in the literature. Early comprehensive reviews, such as Lin et al. [8], established the foundational role of offloading strategies in system performance. However, these earlier works primarily focused on conventional convex optimization and static policies, which often fail in highly dynamic channel environments. Shakarami et al. [14] advanced the field by introducing a taxonomy of ML-driven techniques, including Game Theory and Markov Decision Processes (MDPs). While their study successfully categorizes decision-making criteria, it provides a broad overview of general ML without delving deeply into the architectural nuances of DRL required for high-dimensional state spaces in dense IoT networks.

As the focus shifted toward mobility-aware computing, surveys began addressing Unmanned Aerial Vehicles (UAVs) and Vehicular Edge Computing (VEC). Huda et al. [12] and Abrar et al. [15] investigated UAV-MEC integration, addressing design considerations like mobility management and cost efficiency. Similarly, Ning et al. [16] highlighted the potential of ML in Internet of UAVs (IoUAVs) for industrial automation. Despite these contributions, existing UAV-centric surveys often treat offloading and trajectory control as separate optimization problems or rely on centralized training paradigms that struggle with the distributed nature of swarm-based UAV networks. In the VEC domain, Liu et al. [17] and Wang et al. [18] provided extensive reviews on content caching and data handling. However, a recurring limitation in these domain-specific surveys is the lack of focus on *joint* optimization of communication, computation, and energy constraints under strict real-time inference requirements.

From an algorithmic perspective, recent surveys have attempted to classify optimization techniques. Alali et al. [19] and Walia et al. [20] explored metaheuristic approaches (e.g., genetic algorithms, PSO) and their combination with ML. While metaheuristics are effective for finding global optima in static scenarios, they suffer from slow convergence rates, making them ill-suited for the sub-millisecond decision-making required in modern MEC. Conversely, Zhang et al. [21] addressed stochastic computation offloading, yet their analysis focuses largely on performance bounds rather than practical implementation of DRL agents in non-stationary environments. Furthermore, niche surveys addressing privacy [22, 23] and fault tolerance often discuss these attributes in isolation, neglecting the trade-offs between security overhead and offloading latency.

Table 1 provides a critical comparison of these existing works. While these surveys establish a solid baseline, there remains a critical need for a comprehensive analysis that specifically targets DRL-driven offloading. The existing literature lacks a unified examination of how DRL can overcome the convergence speed of metaheuristics and the adaptability issues of static ML, particularly in heterogeneous, highly dynamic MEC environments.

1.2 Contributions

This review provides critical analysis that extends beyond cataloging existing work. We systematically examine the trade-offs inherent in selecting among DRL algorithmic families, with particular attention to the constraints characteristic of MEC deployments. The contributions of this work include:

- Rigorous comparative analysis of DQN-based and Actor-Critic algorithmic families with respect to computational requirements, convergence behavior, and scalability in resource-constrained MEC environments.
- Explicit mapping of DRL techniques to telecommunications resource management functions including spectrum allocation, bandwidth optimization, and QoS provisioning.
- Integration of MEC offloading research with industry standardization efforts, particularly ETSI MEC specifications and O-RAN architecture.
- Quantitative synthesis of performance results enabling evidence-based algorithm selection for specific deployment scenarios.
- Articulation of precise, actionable research directions addressing identified gaps in current approaches.

The remainder of this paper is structured to provide a logical progression from theoretical foundations to practical implementation. SECTION II establishes the fundamental concepts of MEC, computation offloading, and the DRL paradigms utilized throughout the study. The systematic review methodology is outlined in SECTION III. SECTION IV examines predictive and classification-based DL models, while SECTION V provides a critical taxonomy and analysis of DRL techniques, differentiating between value-based and policy-gradient architectures. SECTION VI synthesizes quantitative performance comparisons across the surveyed literature. SECTION VII bridges the gap between theory and practice by mapping DRL strategies to telecommunications standards (ETSI, O-RAN) and Vehicular Edge Computing (VEC). SECTION VIII addresses practical deployment constraints, including model compression and lightweight architectures. SECTION IX critically evaluates the opti-

Table 1 Analysis of Related Surveys in Multi-Access Edge Computing

Computation Paradigm	Year	Citation	Key Contributions	Limitations & Research Gaps
General MEC & ML	2020	Shakarami [14]	Taxonomy of ML approaches (Game Theory, MDPs) for offloading.	Lacks deep dive into specific DRL architectures (e.g., DDPG vs PPO) for continuous control.
	2022	Li T [22]	Classifies privacy-preserving offloading methods and metrics.	Focuses on privacy overhead without sufficiently addressing the latency trade-off in real-time edge apps.
	2023	Zhang X [21]	Discusses stochastic resource allocation and performance bounds.	Theoretical focus; limited discussion on implementation of model-free learning for unknown channel dynamics.
	2023	Patsias V [24]	Classifies task allocation by network type and DL compression.	Does not extensively cover the energy consumption implications of on-device DL inference.
	2024	Alali S [19]	Taxonomy of metaheuristic approaches for MEC optimization.	Metaheuristics suffer from slow convergence, making them unsuitable for ultra-low latency requirements compared to DRL.
MCC & Cloud	2022	Jin X [23]	Summarizes fault tolerance and privacy in device-pipe-cloud architectures.	Architectures reviewed are often centralized, missing the distinct distributed challenges of MEC.
	2021	Alqarni M.M [25]	Compares AI algorithms on energy, cost, and scalability metrics.	Comparison often treats AI as a black box without analyzing training stability or sample efficiency.
VEC	2021	Liu L [17]	Reviews content caching, data management, and VEC security.	Review of computation offloading is broad; lacks focus on high-mobility handover prediction using DRL.
	2022	Wang M [18]	Analyzes computation offloading work specific to VEC.	Limited discussion on cooperative multi-agent schemes for vehicle platooning.
Integrated Paradigms	2023	Akhlaqi M.Y [26]	Bibliometric and mixed-method synthesis of problem formulations.	High-level statistical analysis; lacks technical critique of the specific optimization algorithms' performance.
	2023	Ning Z [16]	Summarizes MEC/ML applications in IoUAVs (Internet of UAVs).	Focuses heavily on application layers rather than the underlying dynamic resource scheduling algorithms.
	2022	Hou L [27]	Compares MEC with Fog/Cloud regarding industry utilization.	Qualitative comparison; lacks quantitative benchmarking of algorithmic efficiency across paradigms.
UAV-MEC	2022	Huda S [12]	Categorizes offloading strategies by design and mobility features.	Often assumes simplified trajectory models; limited coverage of joint trajectory- offloading control via DRL.
	2021	Abrar M [15]	Summarizes approaches for user-to-UAV task offloading.	Does not adequately address the battery constraints of the UAVs themselves acting as servers.
Fog & Nature-Inspired	2022	Kansal P [28]	Analysis of QoS factors in Fog computing.	Limited scope regarding the integration of AI-driven predictive QoS maintenance.
	2022	Kumar D [29]	Review of nature-inspired algorithms for service placement.	Like [19], overlooks the high computational cost of bio-inspired algorithms in resource-constrained edge nodes.
	2024	Walia G.K [20]	Overview of dynamic resource management using hybrid ML-Metaheuristics.	Hybrid approaches increase complexity; the survey lacks analysis on the feasibility of deployment on lightweight edge devices.

mization trade-offs regarding performance metrics and real-world applications. Finally, SECTION X identifies persisting research gaps and future directions, and the paper concludes with SECTION XI.

2 Conceptual background

2.1 Distinguishing MEC from cloud computing

Understanding the fundamental distinctions between MEC and cloud computing proves essential for appreciating why algorithm selection critera differ substantially between these paradigms. Cloud computing optimizes for aggregate efficiency at scale: centralized data centers achieve economies through resource pooling, statistical multiplexing of workloads, and amortization of infrastructure costs across massive user populations. The optimization objective typically maximizes throughput or minimizes cost per transaction, with latency treated as a secondary concern bounded by service level agreements measured in hundreds of milliseconds [30, 31]. MEC inverts these priorities. Edge deployments optimize for speed, latency, and locality, accepting reduced scale in exchange for proximity to end users [32, 33]. The standard architecture of MEC, structured into three tiers - the cloud layer, edge layer, and user layer, is depicted in Figure 1. Several implications follow from this architectural choice:

Resource Scarcity: Edge servers possess computational capacities orders of magnitude smaller than cloud data centers. A typical MEC server might offer tens of CPU cores and limited GPU resources, compared to thousands of cores available in cloud deployments. This scarcity mandates careful resource allocation and creates scenarios where admission control (deciding which tasks to accept) becomes critical [2].

Heterogeneity: Cloud environments achieve homogeneity through standardization; edge deployments necessarily exhibit heterogeneity reflecting diverse deployment contexts. An edge server at a stadium faces different demand patterns than one serving an industrial facility. This heterogeneity complicates policy learning, as strategies effective in one context may perform poorly in another [34].

Dynamism: Edge environments experience more rapid state changes than cloud deployments. User mobility causes handoffs between edge servers. Wireless channel conditions fluctuate on millisecond timescales. Workload patterns exhibit higher variance due to smaller user populations. These dynamics require policies capable of rapid adaptation [35].

Reliability Constraints: Unlike cloud deployments with extensive redundancy, edge servers may lack backup resources. This constraint influences offloading decisions: the reliability of task completion must factor into optimization alongside latency and energy considerations [36].

Table 2 synthesizes these distinctions, highlighting how they influence algorithm design requirements.

Recent advancements in MEC have focused on overcoming critical challenges, including resource management and orchestration, seamless mobility support, security and privacy enhancements, and the integration of MEC with emerging technologies such as IoT, 5G and edge analytics [37–39]. However, several open challenges remain, particularly the lack of standardized frameworks and the efficient allocation of edge resources [40].

2.2 Computation offloading

A defining capability of MEC is its ability to offload computation-heavy tasks from user devices to edge servers, significantly enhancing battery efficiency and application performance. However, the unpredictably dynamic nature of MEC environments poses considerable impediments for computation offloading, particularly due to device mobility and fluctuations in server availability. To address these obstacles, researchers have developed diverse offloading models and strategies, including parallel task offloading, small-area edge offloading, and DRL-based offloading schemes [10]. The flow of the decision-making process for computation offloading in MEC, as depicted in Figure 2, delineates the execution of tasks, whether they are carried out locally, partially processed at the edge, or entirely offloaded to the cloud, based on the availability of resources, network conditions, and latency constraints.

User devices, such as mobile phones and IoT sensors, generate computational tasks requiring processing resources. The Offloading Decision Maker evaluates the feasibility of local execution, partial offloading, or complete cloud delegation. Tasks awaiting execution in the edge layer are managed through task queues to optimize processing efficiency. In instances where a device possesses adequate resources, the task is carried out locally; conversely, when resources are insufficient, the task is allocated between local computation and edge servers. The decision process is typically modeled by a mixed integer nonlinear programming model [41]. Base stations and edge servers handle offloaded tasks, where base stations facilitate network connectivity between users and the edge infrastructure, and cloud servers process tasks that exceed the computational limits of edge devices.

DL has developed as a powerful enabler for offloading AI-driven computation-intensive tasks in MEC systems [42]. Many AI techniques, including DL-based models, require processing power beyond the capacity of individual edge devices. Edge computing alleviates this challenge by transferring computational workloads to high-performance edge servers, which can support complex learning tasks and location-aware intelligence as an extension of cloud infrastructure. By integrating computation offloading with DL,

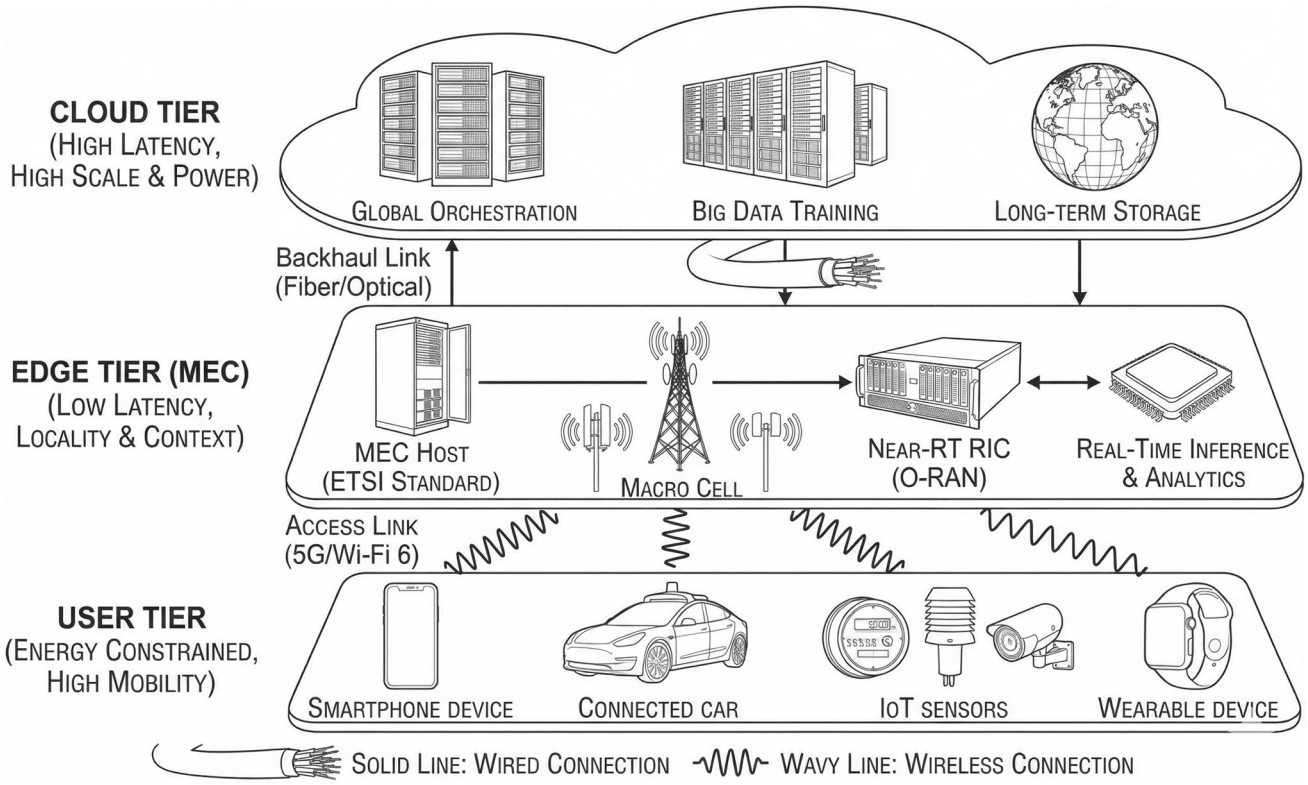

Fig. 1 Standard 3-tier architecture of a typical MEC System

Table 2 Comparative Analysis: MEC versus Cloud Computing Paradigms and Algorithmic Implications

Dimension	Cloud Computing	Multi-Access Edge Computing	Algorithmic Implications
Primary Optimization Objective	Throughput, cost efficiency	Latency, locality	DRL reward functions must prioritize latency terms; discount factors should reflect urgency
Resource Availability	Abundant, elastically scalable	Constrained, finite	Algorithms must handle resource scarcity; action spaces may include admission control
Infrastructure Homogeneity	High (standardized data centers)	Low (diverse deployment contexts)	Transfer learning becomes essential; policies require context adaptation
State Dynamics	Relatively stable	Highly dynamic (mobility, channel variations)	Sample efficiency critical; algorithms must adapt rapidly
Latency Requirements	Hundreds of milliseconds acceptable	Single-digit milliseconds required	Decision-making overhead itself becomes constrained; lightweight inference essential
Failure Handling	Extensive redundancy	Limited backup resources	Risk-aware policies may be necessary; reliability enters reward formulation

MEC systems can enhance mobile device performance, optimize battery life, and enable resource-intensive applications and intelligent services, marking a significant advancement in next-generation smart computing architectures [43].

2.3 Deep reinforcement learning fundamentals

DRL combines deep neural networks, which provide function approximation for high-dimensional state spaces, with reinforcement learning, which provides frameworks for sequential decision-making under uncertainty. The standard formulation models the offloading problem as a Markov

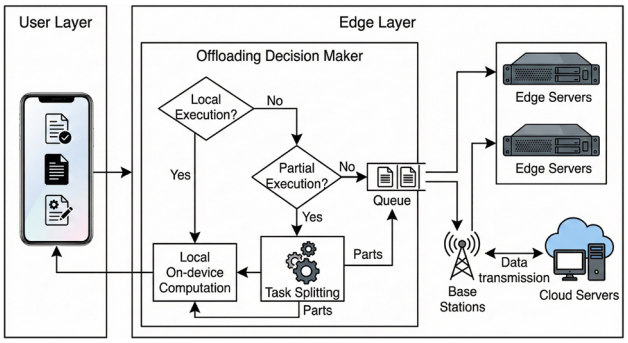

Fig. 2 Computation offloading process in MEC systems

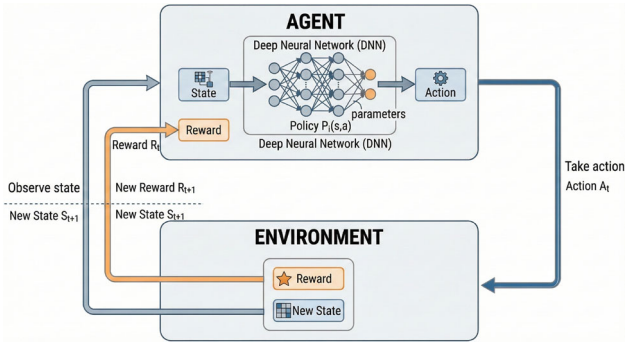

Fig. 3 Agent interaction with environment in DRL paradigm

Decision Process (MDP) defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$ where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $\mathcal{P}$ the transition dynamics, $\mathcal{R}$ the reward function, and $\gamma \in [0, 1)$ the discount factor [13].

In MEC offloading contexts, states typically encode:

- Channel state information (CSI) characterizing wireless link quality
- Queue lengths at edge servers indicating current load
- Task characteristics (computational requirements, data sizes, deadlines)
- Device states (battery levels, local processing capabilities)
- Historical information enabling prediction of future conditions

Figure 3 illustrates an RL system where a DNN estimates the policy function within a DRL framework.

Actions specify offloading decisions: local execution, offloading to specific edge servers, or cloud forwarding. In continuous formulations, actions may additionally specify resource allocation (transmit power, computational resource shares).

The reward function encodes optimization objectives. Common formulations include:

$$r_t = -\alpha \cdot T_{total} - \beta \cdot E_{total} - \lambda \cdot \mathbb{1}_{deadline\_violated} \quad (1)$$

where T_{total} represents total task completion time, E_{total} denotes energy consumption, the indicator function penalizes deadline violations, and weights α, β, λ reflect relative priorities.

Two major algorithmic families dominate DRL applications to MEC offloading:

Value-Based Methods (DQN Family): These approaches learn action-value functions $Q(s, a)$ estimating expected cumulative reward for taking action a in state s. Deep Q-Networks (DQN) [44] approximate this function using neural networks, with innovations including experience replay for sample efficiency and target networks for training stability. Extensions include Double DQN (addressing overestimation bias), Dueling DQN (separating state value from action advantage), and Prioritized Experience Replay (focusing learning on informative transitions).

Policy Gradient Methods (Actor-Critic Family): These approaches directly parameterize and optimize policies $\pi_\theta(a|s)$. Actor-Critic methods combine policy gradient updates (actor) with value function estimation (critic) to reduce variance. Prominent variants include Advantage Actor-Critic (A2C), Asynchronous Advantage Actor-Critic (A3C) enabling parallel training, Proximal Policy Optimization (PPO) providing stable updates through clipped objectives, and Soft Actor-Critic (SAC) incorporating entropy regularization for exploration [45, 46]. The convergence of MEC and DL, particularly in DRL, has enabled a more distributed, efficient, and responsive AI ecosystem at the network edge [47–49].

Table 3 provides a taxonomy of these methods with their key characteristics.

3 Systematic review methodology

This review follows established systematic literature review guidelines adapted for the computer science domain [50]. The methodology comprises four phases: search strategy definition, study selection, data extraction, and synthesis.

3.1 Search strategy

We queried IEEE Xplore, ACM Digital Library, ScienceDirect, and Google Scholar using combinations of terms: ("deep reinforcement learning" OR "DRL" OR "deep Q-network" OR "actor-critic") AND ("mobile edge computing" OR "multi-access edge computing" OR "MEC" OR "edge computing") AND ("offloading" OR "task offloading" OR "computation offloading"). The search covered publications from 2019 through 2025, reflecting the emergence of DRL-based approaches following foundational DQN work.

Table 3 Taxonomy of DRL Algorithms Applied to MEC Offloading

Algorithm	Family	Key Mechanism	Action Space	MEC Suitability
DQN	Value-based	Experience replay, target networks	Discrete	Binary offloading decisions
Double DQN	Value-based	Decoupled action selection/evaluation	Discrete	Reduced overestimation in high-variance MEC states
Dueling DQN	Value-based	Separate value and advantage streams	Discrete	Efficient when many actions yield similar outcomes
A2C/A3C	Actor-Critic	Advantage estimation, parallel workers (A3C)	Discrete or Continuous	Scalable multi-device scenarios
PPO	Actor-Critic	Clipped surrogate objective	Discrete or Continuous	Stable learning under non-stationary MEC dynamics
SAC	Actor-Critic	Maximum entropy framework	Continuous	Joint offloading and resource allocation
TD3	Actor-Critic	Twin critics, delayed policy updates	Continuous	High-dimensional continuous control

3.2 Selection criteria

Inclusion criteria required: (1) peer-reviewed publication in journals or conference proceedings; (2) explicit application of DL or DRL to computation offloading; (3) evaluation in MEC or comparable edge computing contexts; (4) English language. Exclusion criteria eliminated: (1) purely theoretical analyses without evaluation; (2) studies addressing unrelated edge computing aspects (e.g., caching only); (3) duplicate publications reporting identical results.

3.3 Data extraction

For each included study, we extracted: algorithmic approach and variants employed; system model characteristics (number of users, servers, mobility model); optimization objectives and reward formulations; evaluation methodology (simulation, testbed, dataset); reported performance metrics; and comparative baselines.

3.4 Quality assessment

Studies were assessed on: clarity of problem formulation; appropriateness of experimental design; rigor of baseline comparisons; reproducibility (availability of code, detailed parameter specification); and relevance of evaluation metrics to practical deployment. This assessment informs our critical analysis, identifying not merely what approaches exist but which demonstrate methodological soundness.

4 Deep learning techniques for offloading decisions

Before examining DRL approaches, we consider supervised and unsupervised deep learning methods applied to offloading. These approaches typically address prediction tasks that inform offloading decisions rather than directly optimizing policies.

4.1 Predictive models for offloading support

Several works employ deep learning for predicting quantities relevant to offloading decisions:

Workload Prediction: Long Short-Term Memory (LSTM) networks have demonstrated effectiveness in predicting future computational demands based on historical patterns [51]. Such predictions enable proactive resource provisioning at edge servers, reducing the probability that offloaded tasks encounter resource unavailability. Convolutional neural networks (CNNs) have similarly been applied to spatial-temporal workload prediction across geographically distributed edge servers [52].

Channel State Prediction: Wireless channel conditions critically influence offloading decisions: poor channel conditions increase transmission time, potentially negating benefits of edge execution. Deep learning models predicting channel quality enable anticipatory offloading, initiating transmission when favorable conditions are predicted [53]. Recurrent architectures capture temporal correlations in fading channels, while attention mechanisms identify relevant historical observations.

Mobility Prediction: In scenarios involving user mobility, predicting future locations enables preemptive task migration or selection of edge servers along anticipated

trajectories [54]. Graph neural networks have proven particularly effective when mobility occurs over structured environments (road networks, building layouts).

4.2 Deep learning for binary offloading classification

A subset of works frames offloading as a classification problem: given current state, should a task be offloaded or executed locally? Deep neural networks trained on labeled datasets (generated through optimization or simulation) can provide rapid inference [55]. While this approach sacrifices the adaptability of reinforcement learning, it offers advantages in inference speed.

Limitation: Classification approaches require labeled training data reflecting optimal decisions. Generating such data requires solving the optimization problem the classifier aims to avoid, limiting applicability to scenarios where offline optimization is tractable or extensive historical data with known-good decisions exists.

4.3 Autoencoders for state representation

High-dimensional state spaces challenge both learning efficiency and inference latency. Autoencoder architectures compress state representations while preserving information relevant to offloading decisions [56]. Variational autoencoders additionally provide probabilistic representations capturing uncertainty, potentially valuable in MEC environments where state observation is imperfect due to delayed or missing feedback.

5 Deep reinforcement learning techniques: critical analysis

A critical categorization of discrete control methods and hybrid frameworks that integrate control theory with deep learning is presented in Table 4, highlighting their specific stability guarantees and limitations.

5.1 Value-Based methods: DQN and variants

5.1.1 Deep Q-networks in MEC contexts

The application of DQN to MEC offloading was pioneered by works including [35] and [66]. The approach discretizes the action space, typically as binary offloading decisions per task or device, and learns Q-values through temporal difference updates with experience replay and target networks.

Strengths in MEC Contexts:

- *Sample Efficiency through Replay:* Experience replay allows multiple learning updates from each environmental interaction. In MEC, where system access for training may be limited (particularly on operational networks), this efficiency proves valuable.
- *Stable Learning:* Target networks, updated periodically rather than at each step, provide stable regression targets. MEC environments with high state variance benefit from this stabilization.
- *Straightforward Implementation:* DQN's relative simplicity facilitates deployment and debugging, important considerations for practical systems.

Limitations in MEC Contexts:

- *Discrete Action Space Restriction:* DQN fundamentally requires discrete actions. MEC scenarios requiring continuous control (joint offloading and power allocation) require discretization that may yield suboptimal performance or exponentially large action spaces.
- *Overestimation Bias:* Standard DQN tends to overestimate Q-values, potentially leading to suboptimal policies. In MEC, where reward signals may be noisy (due to stochastic channel conditions), this bias can be pronounced.
- *Scalability Challenges:* The action space grows exponentially with the number of devices or tasks requiring simultaneous decisions. With N devices each having K offloading options, the action space contains K^N possibilities, becoming intractable for $N > 10$ in many implementations.

5.1.2 Double DQN: Addressing overestimation

Double DQN [67] decouples action selection from action evaluation, using the online network to select actions and the target network to evaluate them. Several MEC offloading studies adopt this variant [51, 68], reporting reduced overestimation and improved final policy quality.

Critical Assessment: While Double DQN demonstrably reduces overestimation, the improvement magnitude depends on environmental characteristics. In MEC scenarios with deterministic reward components (e.g., fixed computation costs), benefits may be modest. The technique adds minimal computational overhead and should generally be preferred over standard DQN.

5.1.3 Dueling DQN: Value and advantage decomposition

Dueling DQN [69] separates the Q-function into state value $V(s)$ and action advantage $A(s, a)$ components, sharing

Table 4 Value-Based, Lyapunov, and Theory-Integrated Offloading Approaches

Category	Ref	Algorithm	Key Contributions	Critical Analysis
DQN & Q-Learning (Discrete)	[57]	Deep Q-Learning	Reliable task offloading in heterogeneous VEC with multiple nodes.	DQN is limited to discrete action spaces; discretization of continuous variables (e.g., power) leads to the curse of dimensionality.
	[58]	Double DQN	SORA: Reduces processing delays while maintaining V2V link reliability.	Double DQN reduces the overestimation bias of standard DQN, but still struggles with slow convergence in rapidly changing topologies.
	[59]	DQN (JTOBA)	Optimizes total expenses and user action completion time.	Reliance on a single Q-network can lead to instability during training; lacks mechanism for multi-objective trade-offs beyond weighted sums.
	[60]	DDQN (DARLING)	Stochastic computation offloading using Q-function decomposition.	Value decomposition helps, but the approach assumes stationarity in user behavior which is rare in real-world edge scenarios.
	[61]	DDQN + Decision Tree	Intelligent offloading and server selection in Multi-User setups.	Hybridizing with Decision Trees improves interpretability but may limit the model's ability to generalize to unseen network states.
Lyapunov & Optimization Hybrids	[62]	Lyapunov + Actor-Critic	Addresses offloading challenges due to high user mobility in 6G.	Lyapunov optimization provides stability guarantees, but often results in conservative delay performance compared to pure DRL.
	[63]	Lyapunov + DRL	Integrated framework for online stable computation offloading.	The coupling of Lyapunov drift with DRL training can lead to local optima if the penalty weights are not dynamically tuned.
	[64]	MINLP + A3C	Optimization of resource allocation, service caching, and offloading.	MINLP is NP-hard; while A3C approximates the solution, the combinatorial complexity of caching makes real-time inference difficult.
	[65]	SCA + Dinkelbach	OELO: Secure operations in UAV-enabled MEC.	Successive Convex Approximation (SCA) depends heavily on the initial point; integration with Dinkelbach increases computational overhead.

lower network layers. This architecture proves beneficial when action choice has limited impact in many states—the network can accurately estimate state value without precisely differentiating actions.

MEC Applicability: Consider offloading decisions when edge server queues are either empty (any offloading beneficial) or saturated (local execution preferred regardless of task characteristics). In such states, many actions yield similar outcomes; dueling architectures efficiently represent this structure. Studies including [70] confirm improved sample efficiency in MEC scenarios with such characteristics.

5.1.4 Prioritized experience replay

Rather than sampling uniformly from the replay buffer, prioritized experience replay [71] samples transitions proportionally to their temporal difference error magnitude, focusing learning on surprising or informative experiences.

MEC Considerations: Edge environments exhibit heterogeneous transition informativeness. Routine states (moderate load, typical channel conditions) dominate experience; rare states (sudden load spikes, channel outages) are underrepresented yet critical for robust policies. Prioritization

addresses this imbalance. However, prioritization introduces bias correctable through importance sampling weights, adding implementation complexity.

5.2 Policy gradient methods: Actor-critic family

Table 5 examines policy-based and multi-agent architectures, evaluating their efficacy in handling continuous action spaces and decentralized coordination within dynamic edge environments.

5.2.1 Advantage actor-critic (A2C)

A2C maintains separate networks for policy (actor) and value estimation (critic), with the critic providing baseline subtraction to reduce policy gradient variance. The advantage $A(s, a) = Q(s, a) - V(s)$ measures relative action quality, providing more informative learning signals than raw returns.

MEC Applicability: A2C handles continuous action spaces naturally, enabling joint optimization of offloading decisions and resource allocation (transmit power, computational resource shares). Studies including [83, 84] demonstrate effectiveness in multi-resource optimization scenarios intractable for DQN.

5.2.2 Asynchronous advantage actor-critic (A3C)

A3C [85] extends A2C with asynchronous parallel workers, each interacting with independent environment instances and periodically synchronizing with a global model. This parallelism accelerates training and provides diverse experience, reducing correlation that can destabilize learning.

MEC Training Considerations: A3C's parallel architecture aligns well with MEC training scenarios. Multiple edge servers can serve as parallel training environments, with each server's distinct characteristics providing experiential diversity. However, asynchronous updates introduce gradient staleness; workers may update global parameters using gradients computed from outdated policies. In rapidly changing MEC environments, this staleness may be problematic.

5.2.3 Proximal policy optimization (PPO)

PPO [45] constrains policy updates to a trust region, preventing destructively large updates that can occur with unconstrained policy gradients. The clipped surrogate objective:

$$L^{CLIP}(\theta) = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \quad (2)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ denotes the probability ratio, ensures conservative updates.

MEC Advantages: MEC environments are inherently non-stationary: user populations shift, application mixes evolve, and infrastructure changes occur. PPO's conservative updates prevent policy collapse when environmental statistics shift during training. Multiple studies [86–88] report stable convergence in MEC scenarios where other algorithms exhibit oscillation or divergence.

5.2.4 Soft actor-critic (SAC)

SAC [46] incorporates maximum entropy reinforcement learning, augmenting the reward with an entropy bonus that encourages exploration. The objective becomes:

$$J(\pi) = \sum_t \mathbb{E}\left[r(s_t, a_t) + \alpha\mathcal{H}(\pi(\cdot|s_t))\right] \quad (3)$$

where $\mathcal{H}$ denotes entropy and α controls the exploration-exploitation trade-off.

MEC Exploration Benefits: MEC environments can contain regions of state space rarely visited under exploitation-focused policies, such as unusual device configurations, atypical load patterns, and exceptional channel conditions. Entropy regularization ensures continued exploration of such regions, yielding policies robust to distributional shift. SAC additionally employs twin critics (similar to TD3) for stable Q-function estimation.

Computational Overhead: SAC's twin critics and automatic entropy tuning increase computational requirements compared to simpler algorithms. For resource-constrained edge deployment, this overhead may necessitate compromise between algorithmic sophistication and inference efficiency.

5.3 Multi-Agent deep reinforcement learning

MEC scenarios often involve multiple decision-making entities: multiple user devices, multiple edge servers, or combinations thereof. Multi-agent DRL (MADRL) addresses coordination and competition among such entities.

5.3.1 Independent learners

The simplest multi-agent approach treats each agent as independent, learning its own policy while treating other agents as part of the environment. While computationally straightforward, this approach suffers from non-stationarity: each agent's policy changes during training, invalidating other agents' learned models of environmental dynamics [89].

Table 5 Policy-Based, Multi-Agent, and Continuous Control Architectures

Category	Ref	Algorithm	Key Contributions	Critical Analysis
Continuous Control (DDPG/PPO)	[72]	DDPG	Partial offloading for delay-sensitive IIoT to minimize overall delay.	DDPG handles continuous action spaces well but is highly sensitive to hyperparameter tuning and exploration noise.
	[73]	DDPG (MESON)	Joint optimization of performance and dependability in urban VEC.	While effective for continuous power control, DDPG suffers from training instability compared to newer algorithms like TD3 or SAC.
	[74]	PPO2 (Cooperative)	Privacy-oriented offloading for intelligent transport systems.	PPO is more stable than DDPG due to clipped objectives, yet on-policy learning makes it less sample-efficient for real-world edge training.
	[75]	OD-SARSA	Resource management using on-policy updates.	SARSA is safer (conservative) than Q-learning but converges slower, which is detrimental in high-mobility UAV scenarios.
Multi-Agent & Federated (MADRL)	[76]	MAPPO	Addresses computation ambiguities in multi-UAV networks.	Centralized Training Decentralized Execution (CTDE) scales well, but assumes reliable communication for training which may not exist in jammed environments.
	[77]	SAC-based MADRL	Distributed V2V trading and task offloading in vehicular fog.	Soft Actor-Critic (SAC) provides excellent exploration via entropy maximization, but the computational cost of maintaining multiple critics is high for edge nodes.
	[78]	Federated DRL	Charging scheduling and trajectory optimization for UAVs.	Federated averaging reduces privacy leakage but introduces significant communication overhead and struggles with non-IID data across UAVs.
	[79]	MARL + Stochastic Game	Integration of SDN with MEC to mitigate centralized server load.	Stochastic games model competition well but solving for Nash Equilibrium in dynamic networks is computationally prohibitive for real-time apps.
Advanced Architectures (A3C/LSTM)	[80]	A3C	Blockchain throughput optimization in MEC.	Asynchronous updates in A3C speed up training but can lead to "policy lag" where agents update based on obsolete network states.
	[81]	LSTM + Dueling DQN	Time-critical task offloading capturing temporal dependencies.	LSTM captures history effectively but adds significant inference latency compared to simple MLPs, affecting real-time deadline guarantees.
	[82]	MAML (Meta-Learning)	Rapid adaptation to evolving offloading environments (MRLCO).	Meta-learning enables few-shot adaptation to new environments, but the "inner loop" update adds complexity and requires high-quality diverse training data.

5.3.2 Centralized training with decentralized execution (CTDE)

CTDE architectures [90, 91] train agents with access to global information (other agents' states and actions) but execute with only local observations. This framework resolves non-stationarity during training while maintaining practical deployability.

MEC Implementation: During training, a central coordinator (potentially in the cloud) aggregates state information from edge servers and devices, enabling joint policy optimization. Deployed policies execute locally, requiring only local observations. Studies including [92, 93] demonstrate CTDE's effectiveness for multi-user MEC offloading.

5.3.3 Federated reinforcement learning

Federated approaches [94, 95] train DRL models across distributed edge nodes without centralizing raw experience data. Each node trains locally; only model updates (gradients or parameters) are aggregated. This approach addresses privacy concerns; sensitive patterns in user behavior need not leave local edge servers and reduces communication overhead compared to centralizing experience.

Convergence Challenges: Heterogeneity across edge nodes (different user populations, hardware capabilities, network conditions) creates non-IID data distributions that complicate federated aggregation. Techniques from federated learning (FedProx, SCAFFOLD) address these challenges but introduce additional hyperparameters requiring tuning [96].

6 Comparative analysis of algorithmic families

This section synthesizes quantitative comparisons from the surveyed literature, enabling evidence-based algorithm selection for specific deployment scenarios.

6.1 Quantitative performance synthesis

Table 6 aggregates performance metrics across representative studies to highlight distinct algorithmic behaviors. The analysis focuses on Task Success Rate, Average Latency, and Convergence Speed.

6.1.1 Success rate vs. convergence speed

Value-based methods, such as the standard DQN employed by Chen et al. [60], demonstrate a respectable task success rate of 89.2% with moderate convergence speed (approx. 2,500 episodes). However, advanced Policy Gradient methods show superior reliability. For instance, PPO-based approaches [97] achieve a higher success rate of 94.1% and lower latency (32.1 ms). This performance gain is attributed to PPO's clipped objective function, which prevents destructive policy updates in dynamic MEC environments. However, this stability comes at a computational cost; PPO and SAC [81] generally require more training episodes (2,800-4,200) to reach optimal policies compared to Double DQN variants [68] (2,100 episodes).

6.1.2 Latency and energy trade-offs

In scenarios prioritizing energy efficiency, Multi-Agent architectures (MADDPG) [93] perform robustly, maintaining low energy consumption (0.74 J/task) even with dense user scaling (40 users). Conversely, single-agent Actor-Critic methods (A2C) [84] offer competitive latency reduction (35.4 ms) but struggle to maintain energy efficiency as the number of edge servers increases, indicating a need for decentralized coordination in large-scale topologies.

6.1.3 Impact of action space

The data further reveals that algorithms handling continuous action spaces (SAC, DDPG) generally yield lower average latencies (e.g., 33.5 ms for SAC) compared to discrete-action baselines. This suggests that the quantization errors inherent in discretizing power or bandwidth allocation in DQN-based approaches result in sub-optimal resource utilization, confirming the necessity of continuous control architectures for fine-grained MEC optimization.

6.2 Trade-off analysis: DQN vs. actor-critic

Sample Efficiency vs. Final Performance: DQN variants typically achieve reasonable performance with fewer environmental interactions due to experience replay enabling multiple updates per sample. Actor-Critic methods, particularly PPO and SAC, often achieve superior final performance but require more training samples. For MEC deployments where training data collection is expensive (requiring interaction with production systems), DQN's sample efficiency may outweigh Actor-Critic performance advantages.

Computational Complexity vs. Policy Quality: SAC's twin critics and entropy tuning provide robust policies but double critic network computational requirements. Dueling DQN adds architectural complexity but maintains single-network inference. For edge deployment where inference latency is constrained, simpler architectures (DQN, A2C) may be necessary despite reduced policy quality.

Discrete vs. Continuous Control: When offloading decisions are purely binary and resource allocation is predetermined, DQN-family methods suffice and may be preferred

Table 6 Quantitative Performance Comparison Across DRL Algorithms for MEC Offloading

Study	Algorithm	Users	Servers	Task Success (%)	Avg Latency (ms)	Energy (J/task)	Convergence (episodes)
Chen et al. [35]	DQN	10	1	89.2	45.3	0.82	2,500
Li et al. [68]	Double DQN	20	3	91.5	38.7	0.75	2,100
Wang et al. [92]	Dueling DQN	15	2	90.8	41.2	0.78	1,800
Liu et al. [84]	A2C	30	5	92.3	35.4	0.71	3,200
Chen et al. [86]	PPO	25	4	94.1	32.1	0.68	2,800
Huang et al. [87]	PPO	50	8	93.5	36.8	0.72	3,500
Tang et al. [98]	SAC	20	3	93.8	33.5	0.69	4,200
Dinh et al. [93]	MADDPG	40	6	91.2	39.4	0.74	5,100

Table 7 Algorithm Recommendations for Specific MEC Deployment Scenarios

Scenario Characteristics	Recommended Algorithm	Rationale
Single edge server, few users (<15), binary offloading	Double DQN	Sample efficient, stable; overestimation reduction important with small datasets
Multiple servers, moderate users (15-50), binary offloading	Dueling DQN	Architectural efficiency when many states yield similar action preferences
Joint offloading and resource allocation, continuous control	PPO or SAC	Native continuous action support; PPO for stability, SAC for exploration
Highly dynamic environment, frequent distribution shift	PPO	Conservative updates prevent policy collapse under non-stationarity
Multi-device coordination, competitive resources	MADDPG with CTDE	Addresses coordination; centralized training enables joint optimization
Privacy-constrained, distributed training required	Federated A2C/PPO	Model updates only; raw experience remains local
Extreme latency constraints on inference (<1ms)	DQN with small network	Minimizes inference computation; accepts policy quality trade-off

for simplicity. When continuous control is required (power allocation, bandwidth partitioning), Actor-Critic methods (A2C, PPO, SAC) become necessary.

6.3 Scenario-Based algorithm recommendations

Based on synthesized evidence, Table 7 provides scenario-based algorithm recommendations.

6.4 Limitations of existing comparisons

Critical assessment reveals limitations in existing comparative studies:

Inconsistent Baselines: Studies compare against different baselines (random, greedy, optimization-based), complicating cross-study comparison. Standardized baselines would improve comparability.

Simplified Environments: Many studies employ simplified channel models (often Rayleigh fading without spatial correlation), idealized mobility patterns, and homogeneous devices. Real deployments exhibit greater complexity.

Training vs. Deployment Performance: Most studies report training performance; few evaluate deployed policy performance under distribution shift (new user populations, infrastructure changes).

Reproducibility: Limited code availability and incomplete hyperparameter specification hinder reproduction and extension of results.

7 Telecommunications domain integration

This section explicitly maps DRL techniques to telecommunications resource management functions and contextualizes MEC offloading within industry standardization frameworks.

7.1 Mapping DRL to telecom resource management

7.1.1 Spectrum allocation and dynamic spectrum access

Spectrum (the electromagnetic frequency bands over which wireless communication occurs) represents a scarce resource

requiring careful allocation. DRL agents can learn spectrum allocation policies that maximize throughput while respecting interference constraints [99].

Integration with Offloading: Joint spectrum allocation and offloading optimization recognizes that offloading decisions depend on available spectrum (determining achievable transmission rates) while optimal spectrum allocation depends on offloading decisions (determining traffic demands). Studies including [100] demonstrate DRL approaches jointly optimizing these coupled decisions.

7.1.2 Bandwidth management and capacity planning

Bandwidth allocation among users and applications determines achievable data rates and consequently offloading latencies. DRL approaches learn allocation policies optimizing aggregate utility subject to capacity constraints [101].

Channel State Information Processing: Modern wireless systems provide Channel State Information (CSI) indicating link quality. This information serves dual purposes: as state input to DRL agents (informing offloading decisions) and as a target for prediction (enabling anticipatory offloading). Deep learning models predicting CSI evolution enable proactive rather than reactive decisions [53].

7.1.3 Quality of service optimization

QoS specifications such as latency bounds, reliability requirements, and throughput guarantees constrain acceptable offloading outcomes. DRL reward functions incorporate QoS requirements, typically as constraint violation penalties [86]:

$$r = r_{base} - \sum_i \lambda_i \cdot \max(0, g_i(s, a) - c_i) \quad (4)$$

where $g_i(s, a)$ represents the i-th QoS metric achieved under state s and action a, and c_i is the corresponding constraint threshold.

7.2 Industry standards integration

7.2.1 ETSI MEC framework

The ETSI MEC framework [6] defines standardized interfaces for MEC deployments. Key reference points include:

- **Mx1:** Interface between MEC applications and MEC platform, enabling applications to request computational resources
- **Mx2:** Interface between MEC platform and virtualization infrastructure, managing resource allocation
- **Mp1:** Interface exposing platform services to applications, including Radio Network Information Service (RNIS) providing real-time radio condition data

DRL-based offloading can integrate with this framework: RNIS provides state information (channel conditions, cell load); offloading decisions are implemented through Mx1 requests; resource allocation occurs via Mx2. Standardized interfaces ensure interoperability across vendor implementations.

7.2.2 O-RAN architecture compatibility

The Open Radio Access Network (O-RAN) Alliance specifies disaggregated RAN architectures with open interfaces and AI/ML integration [102]. O-RAN defines:

- **RAN Intelligent Controller (RIC):** Platform for hosting AI/ML applications (xApps and rApps) optimizing RAN functions
- **Near-RT RIC:** Enables control loops with latencies of 10ms-1s, suitable for DRL-based resource management
- **Non-RT RIC:** Handles longer-term optimization (>1s), appropriate for policy training and model updates

DRL-based offloading policies can be deployed as xApps on the Near-RT RIC, receiving state information via O-RAN interfaces (E2, A1) and issuing control decisions. This architecture separates training (potentially cloud-based, interfacing via Non-RT RIC) from inference (edge-based, on Near-RT RIC), addressing computational constraints.

7.2.3 5G/6G network implications

Fifth-generation networks introduce capabilities directly relevant to MEC offloading:

- **Network Slicing:** Logical network partitions with guaranteed resources enable QoS-aware offloading where slice selection becomes an additional decision variable [103]
- **Ultra-Reliable Low-Latency Communication (URLLC):** Stringent reliability and latency requirements for industrial and vehicular applications mandate DRL policies explicitly optimizing these metrics
- **Massive Machine-Type Communication (mMTC):** Large device populations require scalable DRL approaches; independent learning with shared policies shows promise [104]

Sixth-generation networks, currently in research phases, envision tighter integration of AI/ML into network operations ("native AI"), sub-millisecond latencies, and tera-

hertz spectrum utilization. These developments will require evolved DRL approaches addressing new frequency characteristics, unprecedented latency constraints, and ubiquitous edge intelligence.

7.3 Vehicular edge computing (VEC) considerations

Vehicular Edge Computing (VEC) extends MEC principles to vehicular networks, with roadside units (RSUs) and vehicle-to-everything (V2X) communication enabling edge services for connected and autonomous vehicles [105, 106]. VEC introduces distinctive challenges:

- **High Mobility:** Vehicles traverse multiple RSU coverage areas rapidly, requiring seamless service continuity and predictive handoff
- **Safety-Critical Applications:** Autonomous driving functions impose extreme reliability requirements; offloading decisions directly impact safety
- **Vehicle-as-Infrastructure:** Parked or slowly moving vehicles can themselves serve as temporary edge nodes, creating dynamic topology

DRL approaches for VEC must incorporate mobility prediction, safety constraints, and dynamic topology awareness. Graph neural networks representing road topology show promise for capturing spatial structure [107].

8 Practical deployment constraints and recent advances

Bridging the gap between research demonstrations and production deployments requires addressing practical constraints often abstracted in academic work.

8.1 Model compression and lightweight architectures

Edge devices and servers possess limited computational resources, constraining model complexity. Techniques addressing this constraint include:

Knowledge Distillation: Training compact "student" networks to mimic larger "teacher" networks transfers policy quality to deployable models [108]. For DRL, policy distillation compresses the actor network while behavior cloning from the teacher provides training signal.

Quantization: Reducing numerical precision from 32-bit floating point to 8-bit integers (or lower) reduces memory footprint and accelerates inference on hardware supporting integer operations. Post-training quantization provides simplicity; quantization-aware training achieves better accuracy.

Pruning: Removing network connections with small weight magnitudes reduces computation. Structured pruning (removing entire channels or layers) provides more predictable speedups than unstructured approaches.

Neural Architecture Search (NAS): Automated search for efficient architectures optimizing accuracy-latency tradeoffs can discover models suited to specific edge hardware [109].

8.2 Real-World deployment challenges

8.2.1 Network latency and communication overhead

DRL agents require state information potentially distributed across devices, edge servers, and network infrastructure. Communication delays in state aggregation introduce decision latency; stale state information degrades policy quality. Approaches addressing this challenge include:

- Local state approximation when global state is unavailable
- Predictive models compensating for communication delays
- Asynchronous decision-making tolerating delayed feedback

8.2.2 Device heterogeneity

Production deployments encompass diverse devices: smartphones spanning multiple generations, IoT sensors with minimal processing capability, laptops, and specialized equipment. A single offloading policy may perform poorly across this heterogeneity. Approaches include:

- Device-type-aware policies with type as state input
- Hierarchical policies with device-specific specialization
- Meta-learning enabling rapid adaptation to new device types

8.2.3 Cold-Start problems

Deploying DRL policies in new environments (new geographic areas, new application mixes, new infrastructure) faces cold-start challenges: policies trained elsewhere may not transfer, yet training from scratch requires extensive interaction.

Transfer learning and domain adaptation techniques address cold-start by leveraging knowledge from source domains [110]. Identifying invariant features across deployments and adapting only deployment-specific components accelerates adaptation.

8.2.4 Safety and robustness

Production systems require robustness to adversarial conditions, sensor failures, and model uncertainty. Safe reinforcement learning approaches constrain policies to avoid catastrophic actions; uncertainty quantification enables conservative decisions when model confidence is low [111].

8.3 Recent advances (2023-2025)

Recent developments address identified limitations:

Federated DRL Integration: Combining federated learning with DRL enables privacy-preserving distributed training. FedRL architectures aggregate policy updates rather than raw experience, addressing privacy concerns in multi-stakeholder MEC deployments [95, 112]. Recent work addresses non-IID data distributions and client heterogeneity through personalization and adaptive aggregation.

Transfer Learning for Rapid Adaptation: Meta-reinforcement learning approaches (MAML, RL^2) enable policies that adapt rapidly to new environments with minimal interaction [113]. For MEC, this enables deployment across diverse contexts without extensive per-context training.

Large Language Model Integration: Emerging work explores integrating Large Language Models (LLMs) with DRL for MEC optimization. LLMs provide semantic understanding of task requirements and natural language interfaces for policy specification, while DRL provides optimization capability [114].

Attention Mechanisms: Transformer architectures and attention mechanisms improve DRL scalability by enabling agents to focus on relevant state components. For MEC with many devices, attention over device states enables effective decision-making without exponential complexity [115].

Hybrid Classical-Quantum Approaches: Quantum computing offers potential advantages for optimization problems. Hybrid approaches combining quantum variational circuits with classical DRL demonstrate promise for combinatorial offloading decisions, though practical quantum hardware limitations currently restrict applications [116].

9 Discussion: Critical analysis of metrics and real-world applications

9.1 Performance metrics: The optimization trade-offs

The evaluation of computation offloading in MEC is fundamentally an exercise in multi-objective optimization, where improvements in one metric often necessitate degradation in others. As illustrated in Figure 4, the landscape of performance indicators is vast, yet the core conflict lies between *service immediacy* (latency) and *resource conservation* (energy/cost).

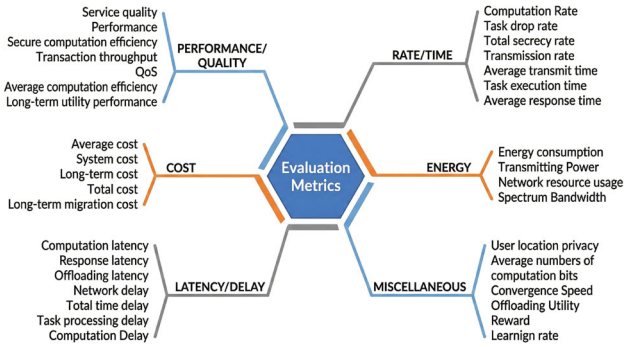

Fig. 4 Comprehensive Taxonomy of Performance Metrics in Edge Computing

9.1.1 The latency-energy pareto frontier

While offloading computationally intensive tasks to edge servers can theoretically minimize processing delay compared to cloud alternatives [126, 127], this benefit is not guaranteed. The trade-off is governed by the communication overhead; if the transmission time exceeds the local computation time, offloading becomes counterproductive. To address this, recent studies have moved beyond simple binary decisions. For instance, Sun et al. [62] and Wang et al. [120] demonstrate that minimizing total latency requires DRL policies that dynamically account for stochastic channel conditions, effectively learning the "break-even point" for offloading. Conversely, energy efficiency presents a different optimization surface. IoT devices leveraging energy harvesting [128] introduce supply-side uncertainty. Strategies proposed by Zhou et al. [129] and Li et al. [130] utilize MDPs and robust optimization to balance energy expenditure against strict latency constraints, revealing that minimizing energy often requires tolerating maximum allowable delays, a critical consideration for battery-constrained UAVs [78].

9.1.2 Cost, rate, and the security tax

Beyond physical metrics, "Cost" in MEC is increasingly modeled as a composite weighted function of energy, delay, and monetary payment for edge resources [121, 123]. Advanced approaches utilizing Lyapunov optimization [63] and Federated DRL [122] successfully decouple these objectives to stabilize queues and minimize long-term system costs. However, a frequently overlooked aspect is the "security tax"—the latency penalty incurred by encryption and privacy-preserving mechanisms. While schemes like ADRLO [118] and A3C-based NOMA [131] enhance data confidentiality and secrecy rates [132, 133], they inevitably

Table 8 Mapping of Offloading Strategies to Key Performance Metrics in MEC

Algorithm/Strategy	Energy	Latency	Computation Rate	Resource Allocation	Scheduling	Security	Privacy	Trajectory	Mobility	Cost
DRL-based SORA [58]		✓	✓		✓	✓	✓			
Deep Q-Learning [57]	✓	✓						✓	✓	
Convex Opt + DQN [117]								✓	✓	
PPO2-based Offloading [74]	✓					✓	✓			
Lyapunov Optimization [63]	✓									✓
A3C Framework [118]	✓		✓		✓	✓	✓			
Binary Offloading [119]			✓		✓					
RL-based RAT [120]								✓	✓	
Deep-SARL [60]			✓							✓
DRLOSM [121]	✓				✓					✓
DROO [119]		✓	✓		✓					
Federated DRL [122]		✓	✓							
DDPG / MAML [72]		✓	✓							
DDQN + DTTOS [61]								✓	✓	
DARLING [60]	✓	✓			✓	✓	✓			✓
Multi-agent DRL [123]	✓	✓	✓				✓			
LSTM Architectures [81]		✓	✓							
Non-Linear Prog. [124]			✓							✓
JTOBA / CAT [59]	✓	✓	✓							✓
CORA / MESON [125]			✓							✓

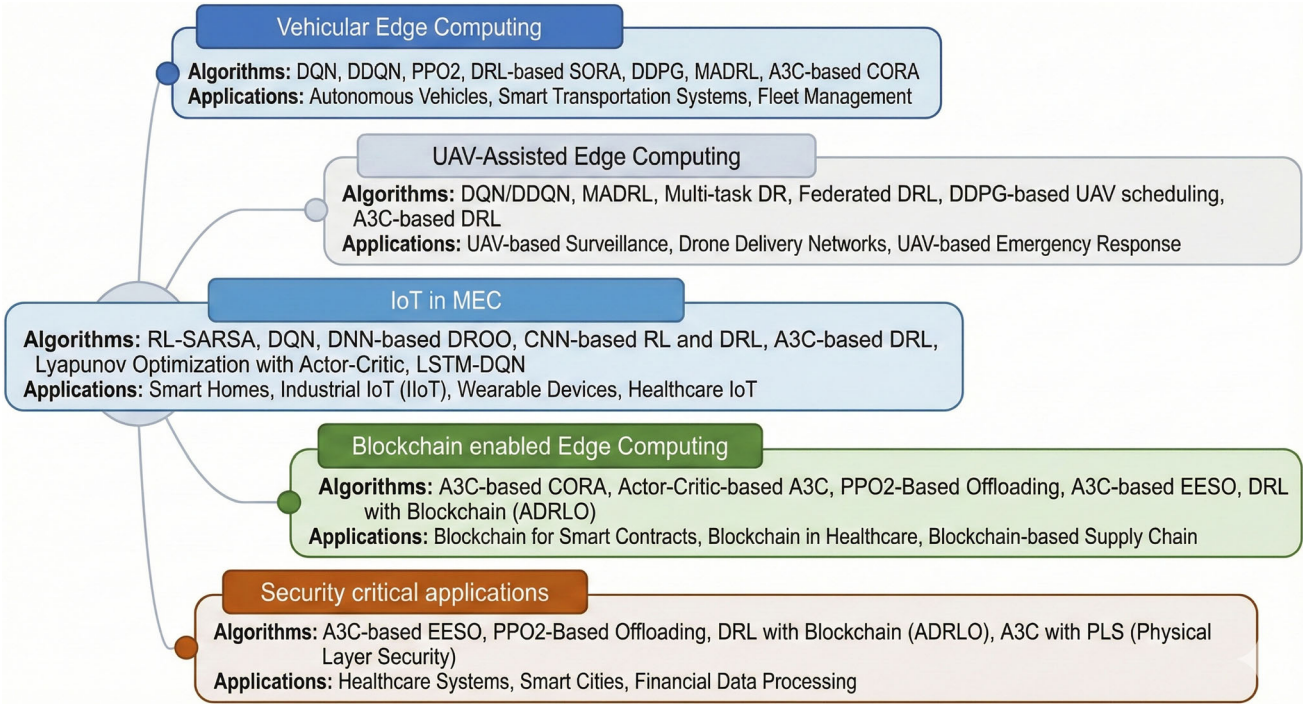

Fig. 5 Mapping of Algorithms to Applications Across Computing Paradigms

reduce the effective computation rate [127]. Optimizing the *effective* throughput, accounting for task drops due to security overhead or buffer overflows [119, 125] remains a persistent challenge in securing real-time systems.

Table 8 provides a critical mapping of these strategies, highlighting how specific algorithms are tailored to distinct metric combinations (e.g., A3C for security-latency balance vs. Lyapunov for cost stability).

9.2 Application domains: Context-aware implementation

The efficacy of DRL algorithms is highly context-dependent. As detailed in Figure 5, real-world deployment requires tailoring algorithms to the specific constraints of the application domain.

9.2.1 Mobility-Centric domains (IoV & UAVs)

In the Internet of Vehicles (IoV) and UAV networks, the primary challenge is not merely offloading, but maintaining service continuity amidst rapid topology changes. Unlike static IoT sensors, vehicular nodes require algorithms that jointly optimize *trajectory control* and *computation offloading*. For instance, Apostolopoulos et al. [134] and Ning et al. [16, 135] highlight that MEC servers in this domain must be mobile (UAV-mounted) to follow the demand. Here, standard RL fails due to state-space explosion; approaches like CORA [125] and Lyapunov-guided energy queues [122] are essential to manage long-term energy budgets while ensuring autonomous driving tasks are processed within milliseconds [136, 137].

9.2.2 Massive connectivity (IIoT & IoT)

For the Industrial IoT (IIoT), the constraint shifts from mobility to *density* and *heterogeneity*. The sheer volume of devices necessitates "partial offloading" strategies, as modeled by Deng et al. [72] using DDPG, where tasks are split to prevent edge server congestion. In these environments, lightweight ML models [138] and multi-purpose DRL [65, 139] are critical to extend battery life by minimizing transmission time [140]. The focus here is on collective resource allocation rather than individual speed.

9.2.3 Trust-Critical domains (Healthcare & Blockchain)

In healthcare (Medical IoT) and blockchain-enabled systems, the objective function is heavily weighted towards integrity and privacy. The integration of blockchain provides decentralized trust [141], but introduces significant computational overhead. Solutions like the ADRLO framework [118] represent a necessary evolution, optimizing the cost of "trust" by using DRL to balance the heavy computation required for consensus mechanisms against the latency sensitivity of medical data monitoring.

Table 9 Research Gaps and Future Directions

Category	Research Gap	Current State & Limitations	Specific Future Directions
Algorithmic Challenges	**Gap 1:** Graph Neural Networks (GNN)	Most DRL approaches utilize flat vector state representations, ignoring the structural relationships among devices, servers, and network topology.	Develop GNN encoders to represent MEC infrastructure as graphs (devices as nodes, links as edges) for topology-agnostic policies. Investigate GNN message-passing for distributed decision-making.
	Gap 2: Privacy-Preserving Multi-Agent Learning	CTDE (Centralized Training Decentralized Execution) assumes global access to agent info, which is problematic for privacy-sensitive, multi-stakeholder environments.	Integrate differential privacy with MADRL to provide formal guarantees. Explore secure multi-party computation for joint training and federated frameworks with provable convergence.
	Gap 3: Robustness Under Adversarial Conditions	DRL policies are highly vulnerable to adversarial perturbations in state observations or reward signals, leading to performance collapse.	Develop adversarially robust DRL specifically for MEC to maintain performance under worst-case perturbations. Create detection mechanisms for attacks on offloading systems.
Implementation Gaps	**Gap 4:** Lightweight Models for Extreme Edge	Current implementations assume servers with substantial resources. Deployment on constrained IoT devices remains largely unexplored.	Develop DRL models for devices with $<$50MB memory and $<$100 MFLOPS budget. Investigate binary neural networks, extreme quantization, and hardware-aware architectures.
	Gap 5: Continual Learning in Non-Stationary Environments	Policies are typically trained once and deployed; adapting to statistical drifts in the environment usually requires full retraining.	Implement continual learning to adapt policies online as statistics drift, utilizing techniques like elastic weight consolidation to prevent catastrophic forgetting.
	Gap 6: Standardized Benchmarking	Disparate simulation environments, traffic models, and metrics across studies preclude meaningful algorithmic comparison.	Establish benchmarks compliant with ETSI MEC specifications. Create open-source platforms with standardized environments and realistic traffic traces for reproducible research.
Emerging Technologies	**Gap 7:** Quantum-Enhanced Optimization	Quantum computing applications for MEC optimization remain largely theoretical and unexplored in practice.	Develop hybrid quantum-classical DRL architectures for near-term devices. Focus on combinatorial offloading sub-problems to demonstrate quantum speedup over classical baselines.
	Gap 8: Semantic Communication Integration	Offloading decisions currently focus on raw data volume, ignoring the semantic importance or meaning of the content.	Integrate semantic communication where transmission depends on information significance. Design DRL agents that jointly optimize semantic extraction and transmission.
	Gap 9: Digital Twin-Enabled Optimization	Training requires interaction with live systems (risky) or simplified simulations (low fidelity).	Develop Digital Twin frameworks for high-fidelity, safe policy training. Investigate Sim-to-Real transfer techniques to minimize performance degradation in production.

In conclusion, there is no "one-size-fits-all" algorithm. VEC demands rapid trajectory adaptation (DQN/MADRL), while secure medical systems require stable, privacy-aware policy gradients (A3C).

10 Research gaps and future directions

Current literature remains largely theoretical, frequently relying on idealized assumptions of consistent connectivity that fail to account for real-world stragglers and intermittent dropouts [82]. To address these practical limitations, research must pivot from static, centralized architectures toward distributed, fault-tolerant frameworks [59] that employ heterogeneous task prioritization [121] and adaptive meta-learning to mitigate dynamic environmental shifts [142]. Existing studies often demonstrate fragility regarding dynamic user churn in serverless contexts [97] and neglect physical layer realities like transmission errors and interference, necessitating rigorous empirical validation and model compression

[81]. Furthermore, decision spaces in current works are often overly simplified; specifically, exclusively continuous models [72] fail to capture the hybrid discrete-continuous nature of IIoT resources, while the restriction to single-cell configurations [131] and the omission of UAV energy replenishment strategies [124] severely compromise the scalability and sustainability of these proposed systems.

Drawing from the critical analysis of the surveyed literature, we have identified key areas where current methodologies fall short of meeting the demands of next-generation MEC systems. These open challenges generally fall into three categories: algorithmic limitations regarding structure and privacy, practical implementation hurdles such as device constraints and benchmarking, and the untapped potential of emerging technologies like quantum computing and semantic communication. A detailed summary of these research gaps, along with specific actionable directions for future investigation, is presented in Table 9.

11 Conclusion

This review has critically examined the transition from heuristic and convex optimization methods to data-driven DRL approaches for computation offloading in MEC networks. Our analysis reveals that while Value-Based methods like DQN offer sample efficiency for discrete decision spaces, the complexity of modern MEC environments increasingly necessitates Policy-Gradient and Actor-Critic architectures (e.g., A3C, SAC) capable of handling continuous, high-dimensional control problems involving joint spectrum, power, and computational resource allocation.

Despite significant algorithmic maturity, a dichotomy remains between simulation-based success and real-world deployability. Current literature often relies on idealized assumptions of persistent connectivity and homogeneous hardware, neglecting the physical realities of intermittent dropouts, straggler devices, and the "security tax" imposed by encryption overhead. Furthermore, the integration of these AI-driven policies within standardized frameworks, such as the O-RAN Near-RT RIC, remains an under-explored frontier.

Moving forward, the evolution of MEC hinges on developing "Green AI" models that are lightweight enough for extreme edge deployment yet robust enough to adapt to non-stationary distributions without catastrophic forgetting. Future research must prioritize the convergence of communication and computation through Semantic Communication and Digital Twin-enabled training, moving beyond simple task offloading toward holistic, privacy-preserving, and autonomous edge intelligence. By addressing these identified implementation gaps, the community can bridge the divide between theoretical potential and the rigorous demands of next-generation telecommunication networks.

Author Contributions S.M. wrote the main manuscript text and prepared figures. CH.V.R.R. reviewed the manuscript.

Funding This research received no external funding.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Ethics, Consent to Participate, and Consent to Publish declarations Not applicable (This study involves no humans or personal data).

Competing interests The authors declare no competing interests.

References

1. Dinh, C. T., Tran, N. H., Nguyen, M. N. H., Hong, C. S., Bao, W., Zomaya, A. Y., & Gramoli, V. (2021). Federated learning over wireless networks: Convergence analysis and resource allocation. *IEEE/ACM Transactions on Networking, 29*(2), 398–409.
2. Mao, Y., You, C., Zhang, J., Huang, K., & Letaief, K. B. (2017). A survey on mobile edge computing: The communication perspective. *IEEE Communications Surveys & Tutorials, 19*(4), 2322–2358.
3. Diao, X., Yang, W., Yang, L., & Cai, Y. (2021). Uav-relaying-assisted multi-access edge computing with multi-antenna base station: Offloading and scheduling optimization. *IEEE Transactions on Vehicular Technology, 70*(9), 9495–9509.
4. Sergi, I., Montanaro, T., Benvenuto, F. L., & Patrono, L. (2021). A smart and secure logistics system based on iot and cloud technologies. *Sensors, 21*(3), 2231.
5. Bittencourt, L., Immich, R., Sakellariou, R., Fonseca, N., Madeira, E., Curado, M., Villas, L., DaSilva, L., Lee, C., & Rana, O. (2018). The internet of things, fog and cloud continuum: Integration and challenges. *Internet of Things, 3–4*(10), 134–155.
6. ETSI, "Mobile edge computing (MEC); framework and reference architecture," ETSI GS MEC 003, vol. 2, no. 1, pp. 1–13, 2019.
7. Mach, P., & Becvar, Z. (2017). Mobile edge computing: A survey on architecture and computation offloading. *IEEE Communications Surveys & Tutorials, 19*(3), 1628–1656.
8. Lin, H., Zeadally, S., Chen, Z., Labiod, H., & Wang, L. (2020). A survey on computation offloading modeling for edge computing. *Journal of Network and Computer Applications, 169*(11), Article 102781.
9. Dubey, S., & Meena, J. (2020). "Computation offloading techniques in mobile edge computing environment: A review," in 2020 International Conference on Communication and Signal Processing (ICCSP), pp. 1217–1223, IEEE, 7.
10. Zhang, H., Yang, Y., Huang, X., Fang, C., & Zhang, P. (2021). Ultra-low latency multi-task offloading in mobile edge computing. *IEEE Access, 9*, 32569–32581.
11. Chen, X., Jiao, L., Li, W., & Fu, X. (2016). Efficient multi-user computation offloading for mobile-edge cloud computing. *IEEE/ACM Transactions on Networking, 24*(5), 2795–2808.
12. Huda, S. A., & Moh, S. (2022). Survey on computation offloading in uav-enabled mobile edge computing. *Journal of Network and Computer Applications, 201*(5), Article 103341.
13. Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction. MIT press, 2nd ed.

14. Shakarami, A., Ghobaei-Arani, M., & Shahidinejad, A. (2020). A survey on the computation offloading approaches in mobile edge computing: A machine learning-based perspective. *Computer Networks, 182*(12), Article 107496.
15. Abrar, M., Ajmal, U., Almohaimeed, Z. M., Gui, X., Akram, R., & Masroor, R. (2021). Energy efficient uav-enabled mobile edge computing for iot devices: A review. *IEEE Access, 9*, 127779–127798.
16. Ning, Z., Hu, H., Wang, X., Guo, L., Guo, S., Wang, G., & Gao, X. (2024). Mobile edge computing and machine learning in the internet of unmanned aerial vehicles: A survey. *ACM Computing Surveys, 56*(1), 1–31.
17. Liu, L., Chen, C., Pei, Q., Maharjan, S., & Zhang, Y. (2021). Vehicular edge computing and networking: A survey. *Mobile Networks and Applications, 26*(6), 1145–1168.
18. Wang, M., Yi, H., Jiang, F., Lin, L., & Gao, M. (2022). Review on offloading of vehicle edge computing. *Journal of Artificial Intelligence and Technology,8*.
19. Alali, S., & Assalem, A. (2023). Metaheuristics method for computation offloading in mobile edge computing: Survey. *Journal of Advanced Research in Applied Sciences and Engineering Technology, 36*(12), 43–73.
20. Walia, G. K., Kumar, M., & Gill, S. S. (2024). Ai-empowered fog/edge resource management for iot applications: A comprehensive review, research challenges, and future perspectives. *IEEE Communications Surveys & Tutorials, 26*(1), 619–669.
21. Zhang, X., & Debroy, S. (2023). Resource management in mobile edge computing: A comprehensive survey. *ACM Computing Surveys, 55*(12), 1–37.
22. Li, T., He, X., Jiang, S., & Liu, J. (2022). A survey of privacy-preserving offloading methods in mobile-edge computing. *Journal of Network and Computer Applications, 203*(7), Article 103395.
23. Jin, X., Hua, W., Wang, Z., & Chen, Y. (2022). A survey of research on computation offloading in mobile cloud computing. *Wireless Networks, 28*(5), 1563–1585.
24. Patsias, V., Amanatidis, P., Karampatzakis, D., Lagkas, T., Michalakopoulou, K., & Nikitas, A. (2023). Task allocation methods and optimization techniques in edge computing: A systematic review of the literature. *Future Internet, 15*(7), 254.
25. Alqarni, M. M., Cherif, A., & Alkayal, E. (2021). A survey of computational offloading in cloud/edge-based architectures: Strategies, optimization models and challenges. *KSII Transactions on Internet and Information Systems, 15*, 952–973.
26. Akhlaqi, M. Y., & Hanapi, Z. B. M. (2023). Task offloading paradigm in mobile edge computing-current issues, adopted approaches, and future directions. *Journal of Network and Computer Applications, 212*(3), Article 103568.
27. Hou, L., Gregory, M. A., & Li, S. (2022). A survey of multi-access edge computing and vehicular networking. *IEEE Access, 10*, 123436–123451.
28. Kansal, P., Kumar, M., & Verma, O. P. (2022). Classification of resource management approaches in fog/edge paradigm and future research prospects: a systematic review. *The Journal of Supercomputing, 78*(7), 13145–13204.
29. Kumar, D., Baranwal, G., Shankar, Y., & Vidyarthi, D. P. (2022). A survey on nature-inspired techniques for computation offloading and service placement in emerging edge technologies. *World Wide Web, 25*(9), 2049–2107.
30. Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., et al. (2010). A view of cloud computing. *Communications of the ACM, 53*(4), 50–58.
31. Jiang, T., Chen, Z., Zhao, Z., Feng, M., & Zhou, J. (2024). Deep reinforcement learning based task offloading and resource allocation in mobile edge computing network with heterogeneous tasks. *IEEE Internet of Things Journal,* pp. 1–1.
32. Kim, T., Al-Tarazi, M., Lin, J.-W., & Choi, W. (2021). Optimal container migration for mobile edge computing: Algorithm, system design and implementation. *IEEE Access, 9*, 158074–158090.
33. Cicconetti, C., Conti, M., & Passarella, A. (2020). Architecture and performance evaluation of distributed computation offloading in edge computing. *Simulation Modelling Practice and Theory, 101*(5), Article 102007.
34. Wang, X., Han, Y., Leung, V. C., Niyato, D., Yan, X., & Chen, X. (2020). Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials, 22*(2), 869–904.
35. Chen, X., Zhang, H., Wu, C., Mao, S., Ji, Y., & Bennis, M. (2019). Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning. *IEEE Internet of Things Journal, 6*(3), 4005–4018.
36. Tran, T. X., & Pompili, D. (2019). Task offloading in mobile edge computing with deep reinforcement learning. *IEEE International Conference on Autonomic Computing (ICAC),* pp. 151–157.
37. Yu, H., & Yang, J. (2024). "Deep reinforcement learning offloading for computation cost minimization of wireless powered mobile edge computing," in Proceedings of the 2024 International Academic Conference on Edge Computing, Parallel and Distributed Computing, ECPDC '24, (New York, NY, USA), p. 1–8, Association for Computing Machinery.
38. Wang, Y., Wu, H., & Li, R. (2024). Deep graph reinforcement learning for mobile edge computing: Challenges and solutions. *IEEE Network, 38*(5), 314–323.
39. Jin, X., Zhang, S., Ding, Y., & Wang, Z. (2024). Task offloading for multi-server edge computing in industrial internet with joint load balance and fuzzy security. *Scientific Reports, 14*, 27813.
40. Varma, G. P. S., & Indukurib, H. (2023). Mobile edge computing architecture challenges, applications, and future directions. *International Journal of Grid and High Performance Computing, 15*(1), 1–23.
41. Qin, Y., Chen, J., Jin, L., Yao, R., & Gong, Z. (2025). Task offloading optimization in mobile edge computing based on a deep reinforcement learning algorithm using density clustering and ensemble learning. *Scientific Reports, 15*, 211.
42. Fang, J., Qu, D., Chen, H., & Liu, Y. (2024). Dependency-aware dynamic task offloading based on deep reinforcement learning in mobile-edge computing. *IEEE Transactions on Network and Service Management, 21*(2), 1403–1415.
43. Gao, H., Wang, X., Wei, W., Al-Dulaimi, A., & Xu, Y. (2024). Com-ddpg: Task offloading based on multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the internet of vehicles. *IEEE Transactions on Vehicular Technology, 73*(1), 348–361.
44. Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., et al. (2015). Human-level control through deep reinforcement learning. *Nature, 518*(7540), 529–533.
45. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347.
46. Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in International Conference on Machine Learning (ICML), pp. 1861–1870, PMLR.
47. Cui, W. (2020). Research and application of edge computing based on deep learning. *Journal of Physics: Conference Series, 1646*(9), Article 012016.
48. Zhang, M., Zhang, F., Lane, N. D., Shu, Y., Zeng, X., Fang, B., Yan, S., & Xu, H. (2020). *Deep learning in the era of edge computing: Challenges and opportunities*.

49. Joshi, P., Hasanuzzaman, M., Thapa, C., Afli, H., & Scully, T. (2023). Enabling all in-edge deep learning: A literature review. *IEEE Access, 11*, 3431–3460.
50. Kitchenham, B. (2004). Procedures for performing systematic reviews. *Keele, UK, Keele University, 33*(2004), 1–26.
51. Chen, X., Li, Z., Chu, Y., & Chockalingam, A. (2021). Deep reinforcement learning for internet of things: A comprehensive survey. *IEEE Communications Surveys & Tutorials, 23*(3), 1659–1692.
52. Xu, Y., Ren, J., Zhang, G., Zhang, C., & Crowcroft, J. (2019). Deep reinforcement learning based computation offloading and resource allocation in MEC, in IEEE Wireless Communications and Networking Conference (WCNC), pp. 1–6, IEEE.
53. Luo, C., Ji, J., Wang, Q., Chen, X., & Li, P. (2020). Channel state information prediction for 5G wireless communications: A deep learning approach. *IEEE Transactions on Network Science and Engineering, 7*(1), 227–236.
54. Wang, J., Hu, J., Min, G., Zomaya, A. Y., & Georgalas, N. (2019). Mobility-aware task offloading and migration schemes in edge computing networks. *IEEE Access, 7*, 34360–34371.
55. Huang, L., Feng, X., Feng, A., Huang, Y., & Qian, L. P. (2019). Deep reinforcement learning for computation offloading and resource allocation in MEC. *IEEE Internet of Things Journal, 6*(4), 5838–5849.
56. Ndikumana, A., Tran, N. H., Ho, T. M., Hong, C. S., Han, Z., & Niyato, D. (2019). Deep learning based hierarchical task prediction for joint optimization of computation offloading and resource allocation in MEC. *IEEE Transactions on Industrial Informatics, 16*(5), 3151–3162.
57. Zhang, K., Zhu, Y., Leng, S., He, Y., Maharjan, S., & Zhang, Y. (2019). Deep learning empowered task offloading for mobile edge computing in urban informatics. *IEEE Internet of Things Journal, 6*(10), 7635–7647.
58. Ju, Y., Chen, Y., Cao, Z., Liu, L., Pei, Q., Xiao, M., Ota, K., Dong, M., & Leung, V. C. M. (2023). Joint secure offloading and resource allocation for vehicular edge computing network: A multi-agent deep reinforcement learning approach. *IEEE Transactions on Intelligent Transportation Systems, 24*(5), 5555–5569.
59. Huang, L., Feng, X., Zhang, C., Qian, L., & Wu, Y. (2019). Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing. *Digital Communications and Networks, 5*(2), 10–17.
60. Chen, X., Zhang, H., Wu, C., Mao, S., Ji, Y., & Bennis, M. (2019). Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning. *IEEE Internet of Things Journal, 6*(6), 4005–4018.
61. Liu, T., Tang, L., Wang, W., Chen, Q., & Zeng, X. (2022). Digital-twin-assisted task offloading based on edge collaboration in the digital twin edge network. *IEEE Internet of Things Journal, 9*(1), 1427–1444.
62. Sun, W., Zhang, H., Wang, R., & Zhang, Y. (2020). Reducing offloading latency for digital twin edge networks in 6g. *IEEE Transactions on Vehicular Technology, 69*(10), 12240–12251.
63. Bi, S., Huang, L., Wang, H., & Zhang, Y.-J.A. (2021). Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge computing networks. *IEEE Transactions on Wireless Communications, 20*(11), 7519–7537.
64. Zhou, H., Wang, Z., Zheng, H., He, S., & Dong, M. (2023). Cost minimization-oriented computation offloading and service caching in mobile cloud-edge computing: An a3c-based approach. *IEEE Transactions on Network Science and Engineering, 10*(5), 1326–1338.
65. Ding, Y., Feng, Y., Lu, W., Zheng, S., Zhao, N., Meng, L., Nallanathan, A., & Yang, X. (2023). Online edge learning offloading and resource management for uav-assisted mec secure communications. *IEEE Journal of Selected Topics in Signal Processing, 17*(1), 54–65.
66. Min, M., Xiao, L., Chen, Y., Cheng, P., Wu, D., & Zhuang, W. (2019). Learning-based computation offloading for IoT devices with energy harvesting. *IEEE Transactions on Vehicular Technology, 68*(2), 1930–1941.
67. Van Hasselt, H., Guez, A., & Silver, D. (2016). Deep reinforcement learning with double Q-learning. *Proceedings of the AAAI Conference on Artificial Intelligence, 30*, 2094–2100.
68. Li, J., Gao, H., Lv, T., & Lu, Y. (2020). Deep reinforcement learning based computation offloading and resource allocation for MEC. *IEEE Wireless Communications and Networking Conference (WCNC),* pp. 1–6.
69. Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., & Freitas, N. (2016). Dueling network architectures for deep reinforcement learning, in International Conference on Machine Learning (ICML), pp. 1995–2003, PMLR.
70. Zhan, W., Luo, C., Wang, J., Wang, C., Min, G., Duan, H., & Zhu, Q. (2020). Deep reinforcement learning based offloading and resource allocation in vehicular edge computing. *IEEE Transactions on Vehicular Technology, 69*(11), 14298–14311.
71. Schaul, T., Quan, J., Antonoglou, I., & Silver, D. (2016). Prioritized experience replay. in *International Conference on Learning Representations (ICLR)*.
72. Deng, X., Yin, J., Guan, P., Xiong, N. N., Zhang, L., & Mumtaz, S. (2023). Intelligent delay-aware partial computing task offloading for multiuser industrial internet of things through edge computing. *IEEE Internet of Things Journal, 10*(2), 2954–2966.
73. Zhao, L., Zhang, E., Wan, S., Hawbani, A., Al-Dubai, A. Y., Min, G., & Zomaya, A. Y. (2024). Meson: A mobility-aware dependent task offloading scheme for urban vehicular edge computing. *IEEE Transactions on Mobile Computing, 23*(5), 4259–4272.
74. Gao, H., Huang, W., Liu, T., Yin, Y., & Li, Y. (2023). Ppo2: Location privacy-oriented task offloading to edge computing using reinforcement learning for intelligent autonomous transport systems. *IEEE Transactions on Intelligent Transportation Systems, 24*(7), 7599–7612.
75. Alfakih, T., Hassan, M. M., Gumaei, A., Savaglio, C., & Fortino, G. (2020). Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on sarsa. *IEEE Access, 8*, 54074–54084.
76. Li, B., & Shan, H. (2024). Offloading revenue maximization in multi-uav-assisted mobile edge computing for video stream. *IEEE Internet of Things Journal,* pp. 1–1.
77. Wei, Z., Li, B., Zhang, R., Cheng, X., & Yang, L. (2024). Many-to-many task offloading in vehicular fog computing: A multi-agent deep reinforcement learning approach. *IEEE Transactions on Mobile Computing, 23*(3), 2107–2122.
78. Wang, X., Li, J., Ning, Z., Song, Q., Guo, L., & Jamalipour, A. (2024). Wireless powered metaverse: Joint task scheduling and trajectory design for multi-devices and multi-uavs. *IEEE Journal on Selected Areas in Communications, 42*(3), 552–569.
79. Wu, G., Wang, H., Zhang, H., Zhao, Y., Yu, S., & Shen, S. (2023). Computation offloading method using stochastic games for software-defined-network-based multiagent mobile edge computing. *IEEE Internet of Things Journal, 10*(10), 17620–17634.
80. Feng, J., Yu, F. R., Pei, Q., Chu, X., Du, J., & Zhu, L. (2020). Cooperative computation offloading and resource allocation for blockchain-enabled mobile-edge computing: A deep reinforcement learning approach. *IEEE Internet of Things Journal, 7*(7), 6214–6228.
81. Tang, M., & Wong, V. W. (2022). Deep reinforcement learning for task offloading in mobile edge computing systems. *IEEE Transactions on Mobile Computing, 21*(6), 1985–1997.
82. Wang, J., Hu, J., Min, G., Zomaya, A. Y., & Georgalas, N. (2021). Fast adaptive task offloading in edge computing based on meta reinforcement learning. *IEEE Transactions on Parallel and Distributed Systems, 32*(1), 242–253.

83. Wang, F., Xu, J., Wang, X., & Cui, S. (2019). Computation offloading and resource allocation in mobile edge computing with multiple independent tasks. *IEEE Transactions on Communications, 67*(8), 5391–5408.
84. Liu, Y., Yan, J., & Zhao, X. (2020). Deep reinforcement learning based latency minimization for mobile edge computing with virtualization in maritime UAV communication network. *IEEE Transactions on Vehicular Technology, 69*(4), 4225–4234.
85. Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T., Harley, T., Silver, D., & Kavukcuoglu, K. (2016). Asynchronous methods for deep reinforcement learning, in International Conference on Machine Learning (ICML), pp. 1928–1937, PMLR.
86. Chen, Z., & Wang, X. (2021). Deep reinforcement learning for computation offloading in mobile edge computing environment. *Computer Communications, 175*, 1–12.
87. Huang, L., Bi, S., & Zhang, Y.-J.A. (2020). Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks. *IEEE Transactions on Mobile Computing, 19*(11), 2581–2593.
88. Alfakih, T., Hassan, M. M., Gumaei, A., Savaglio, C., & Fortino, G. (2020). Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on SARSA. *IEEE Access, 8*, 54074–54084.
89. Tan, M. (1993). Multi-agent reinforcement learning: Independent vs. cooperative agents," in Proceedings of the Tenth International Conference on Machine Learning, pp. 330–337.
90. Lowe, R., Wu, Y. I., Tamar, A., Harb, J., Abbeel, P., & Mordatch, I. (2017). Multi-agent actor-critic for mixed cooperative-competitive environments, in Advances in Neural Information Processing Systems (NeurIPS), *30*.
91. Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., & Whiteson, S. (2018). Counterfactual multi-agent policy gradients, in Proceedings of the AAAI Conference on Artificial Intelligence, vol. *32*.
92. Wang, J., Hu, J., Min, G., Zhan, W., Zomaya, A., & Georgalas, N. (2022). Learning based mobility management under uncertainties for mobile edge computing. *IEEE Transactions on Mobile Computing, 21*(3), 943–957.
93. Dinh, T. Q., Tang, J., La, Q. D., & Quek, T. Q. (2017). Offloading in mobile edge computing: Task allocation and computational frequency scaling. *IEEE Transactions on Communications, 65*(8), 3571–3584.
94. Zhuo, H. H., Feng, W., Xu, Q., Yang, Q., & Lin, Y. (2019). Federated reinforcement learning, in arXiv preprint arXiv:1901.08277.
95. Wang, X., Han, Y., Wang, C., Zhao, Q., Chen, X., & Chen, M. (2020). Federated deep reinforcement learning for internet of things with decentralized cooperative edge caching. *IEEE Internet of Things Journal, 7*(10), 9441–9455.
96. Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. *Proceedings of Machine Learning and Systems, 2*, 429–450.
97. Chen, J., Xing, H., Xiao, Z., Xu, L., & Tao, T. (2021). A drl agent for jointly optimizing computation offloading and resource allocation in mec. *IEEE Internet of Things Journal, 8*(12), 17508–17524.
98. Tang, M., & Wong, V. W. (2022). Deep reinforcement learning based task offloading and resource allocation in MEC-enabled IoT networks. *IEEE Internet of Things Journal, 9*(6), 4193–4205.
99. Naparstek, O., & Cohen, K. (2019). Deep multi-user reinforcement learning for dynamic spectrum access in multichannel wireless networks. *IEEE Transactions on Wireless Communications, 18*(1), 310–323.
100. He, Y., Zhai, D., Zhang, F., Tang, D., Du, R., Wang, R., & Yu, F. R. (2020). Joint computation offloading and resource allocation for mobile edge computing with deep reinforcement learning. *IEEE Internet of Things Journal, 8*(15), 12086–12099.
101. Xu, Y., Ren, J., Zhang, G., Zhang, C., & Crowcroft, J. (2020). Deep reinforcement learning based computation offloading and resource allocation for MEC. *IEEE Wireless Communications and Networking Conference (WCNC),* pp. 1–6.
102. O-RAN Alliance, "O-ran minimum viable plan and commercialization strategy," June (2021). White Paper.
103. Zhang, K., Zhu, Y., Leng, S., He, Y., Maharjan, S., & Zhang, Y. (2020). Deep learning based service slicing and offloading for 5G-enabled mobile edge computing. *IEEE Transactions on Mobile Computing, 19*(11), 2563–2574.
104. Nasir, Y. S., & Guo, D. (2020). Deep reinforcement learning for wireless access in massive MTC. *IEEE Transactions on Wireless Communications, 19*(12), 8204–8218.
105. Liu, J., Wan, J., Jia, B., Zeng, J., Guo, D., Hassan, M. M., & Alamri, A. (2019). Vehicular edge computing and networking: A survey. *Mobile Networks and Applications, 24*(4), 1198–1212.
106. Zhang, J., & Letaief, K. B. (2019). Vehicle-to-vehicle communications: A new frontier for machine learning. *IEEE Vehicular Technology Magazine, 14*(2), 20–30.
107. Zhou, J., Cui, G., Hu, S., Zhang, Z., Yang, C., Liu, Z., Wang, L., Li, C., & Sun, M. (2020). Graph neural networks: A review of methods and applications. *AI Open, 1*, 57–81.
108. Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the knowledge in a neural network, arXiv preprint arXiv:1503.02531.
109. Tan, M., Chen, B., Pang, R., Vasudevan, V., Sandler, M., Howard, A., & Le, Q. V. (2019). Mnasnet: Platform-aware neural architecture search for mobile. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition,* pp. 2820–2828.
110. Zhu, Z., Lin, K., & Jain, A. K. (2023). Transfer learning in deep reinforcement learning: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 45*(11), 13344–13362.
111. García, J., & Fernández, F. (2015). A comprehensive survey on safe reinforcement learning. *Journal of Machine Learning Research, 16*(1), 1437–1480.
112. Qi, Q., & Wang, X. (2021). Federated learning for computation offloading in MEC. *IEEE Wireless Communications Letters, 10*(8), 1672–1676.
113. Finn, C., Abbeel, P., & Levine, S. (2017). Model-agnostic meta-learning for fast adaptation of deep networks, in International Conference on Machine Learning (ICML), pp. 1126–1135, PMLR.
114. Wu, Y., Wang, Y., & Li, Y. (2023). Large language models for edge networking: A survey, arXiv preprint arXiv:2311.01123.
115. Chen, X., & Jiao, L. (2023). Attention-based deep reinforcement learning for computation offloading in MEC with time-varying wireless channels. *IEEE Transactions on Mobile Computing, 22*(4), 2132–2145.
116. Chen, X., & Zhang, J. (2022). Quantum deep reinforcement learning for computation offloading in mobile edge computing. *IEEE Journal on Selected Areas in Communications, 40*(2), 457–470.
117. Wang, L., Wang, K., Pan, C., Xu, W., Aslam, N., & Nallanathan, A. (2022). Deep reinforcement learning based dynamic trajectory control for uav-assisted mobile edge computing. *IEEE Transactions on Mobile Computing, 21*(10), 3536–3550.
118. He, Q., Feng, Z., Fang, H., Wang, X., Zhao, L., Yao, Y., & Yu, K. (2023). A blockchain-based scheme for secure data offloading in healthcare with deep reinforcement learning. *IEEE/ACM Transactions on Networking,* pp. 1–16.
119. Huang, L., Bi, S., & Zhang, Y.-J.A. (2020). Deep reinforcement learning for online computation offloading in wireless powered mobile-edge computing networks. *IEEE Transactions on Mobile Computing, 19*(11), 2581–2593.
120. Wang, J., Hu, J., Min, G., Zhan, W., Ni, Q., & Georgalas, N. (2019). Computation offloading in multi-access edge computing

using a deep sequential model based on reinforcement learning. *IEEE Communications Magazine, 57*(5), 64–69.
121. Zhan, W., Luo, C., Wang, J., Wang, C., Min, G., Duan, H., & Zhu, Q. (2020). Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing. *IEEE Internet of Things Journal, 7*(6), 5449–5465.
122. Dai, X., Xiao, Z., Jiang, H., & Lui, J. C. S. (2024). Uav-assisted task offloading in vehicular edge computing networks. *IEEE Transactions on Mobile Computing, 23*(4), 2520–2534.
123. Zhao, N., Ye, Z., Pei, Y., Liang, Y.-C., & Niyato, D. (2022). Multi-agent deep reinforcement learning for task offloading in uav-assisted mobile edge computing. *IEEE Transactions on Wireless Communications, 21*(9), 6949–6960.
124. Lin, N., Bai, L., Hawbani, A., Guan, Y., Mao, C., Liu, Z., & Zhao, L. (2024). Deep-reinforcement-learning-based computation offloading for servicing dynamic demand in multi-uav-assisted iot network. *IEEE Internet of Things Journal, 11*(5), 17249–17263.
125. Huang, J., Wan, J., Lv, B., Ye, Q., & Chen, Y. (2023). Joint computation offloading and resource allocation for edge-cloud collaboration in internet of vehicles via deep reinforcement learning. *IEEE Systems Journal, 17*(6), 2500–2511.
126. Ale, L., Zhang, N., King, S. A., & Guardiola, J. (2021). Spatio-temporal bayesian learning for mobile edge computing resource planning in smart cities. *ACM Transactions on Internet Technology, 21*(8), 1–21.
127. Abednego Acheampong, X. X., & Zhang, Y. (2023). A review of the current task offloading algorithms, strategies and approach in edge computing systems. *Computer Modeling in Engineering & Sciences, 134*(1), 35–88.
128. Min, M., Wan, X., Xiao, L., Chen, Y., Xia, M., Wu, D., & Dai, H. (2019). Learning-based privacy-aware offloading for healthcare iot with energy harvesting. *IEEE Internet of Things Journal, 6*(6), 4307–4316.
129. Zhou, H., Jiang, K., Liu, X., Li, X., & Leung, V. C. M. (2022). Deep reinforcement learning for energy-efficient computation offloading in mobile-edge computing. *IEEE Internet of Things Journal, 9*(1), 1517–1530.
130. Li, B., Yang, R., Liu, L., Wang, J., Zhang, N., & Dong, M. (2024). Robust computation offloading and trajectory optimization for multi-uav-assisted mec: A multiagent drl approach. *IEEE Internet of Things Journal, 11*(2), 4775–4786.
131. Ju, Y., Cao, Z., Chen, Y., Liu, L., Pei, Q., Mumtaz, S., Dong, M., & Guizani, M. (2024). Noma-assisted secure offloading for vehicular edge computing networks with asynchronous deep reinforcement learning. *IEEE Transactions on Intelligent Transportation Systems,25*(3), 2627–2640.
132. Jiang, X. (2022). Energy consumption optimization of edge computing based on reinforcement learning. *Journal of Physics: Conference Series, 2246*(4), Article 012076.
133. Premkumar, S., & Sigappi, A. (2018). A survey of architecture, framework and algorithms for resource management in edge computing. *EAI Endorsed Transactions on Energy Web, 7*, Article 167788.
134. Apostolopoulos, P. A., Fragkos, G., Tsiropoulou, E. E., & Papavassiliou, S. (2023). Data offloading in uav-assisted multi-access edge computing systems under resource uncertainty. *IEEE Transactions on Mobile Computing, 22*(1), 175–190.
135. Ning, Z., Yang, Y., Wang, X., Guo, L., Gao, X., Guo, S., & Wang, G. (2023). Dynamic computation offloading and server deployment for uav-enabled multi-access edge computing. *IEEE Transactions on Mobile Computing, 22*(5), 2628–2644.
136. Liu, L., Zhao, M., Yu, M., Jan, M. A., Lan, D., & Taherkordi, A. (2022). Mobility-aware multi-hop task offloading for autonomous driving in vehicular edge computing and networks. *IEEE Transactions on Intelligent Transportation Systems,* pp. 1–14.
137. Chai, F., Zhang, Q., Yao, H., Xin, X., Gao, R., & Guizani, M. (2023). Joint multi-task offloading and resource allocation for mobile edge computing systems in satellite iot. *IEEE Transactions on Vehicular Technology, 72*(6), 7783–7795.
138. Merenda, M., Porcaro, C., & Iero, D. (2020). Edge machine learning for ai-enabled iot devices: A review. *Sensors, 20*(4), 2533.
139. Yang, L., Yao, H., Wang, J., Jiang, C., Benslimane, A., & Liu, Y. (2020). Multi-uav-enabled load-balance mobile-edge computing for iot networks. *IEEE Internet of Things Journal, 7*(8), 6898–6908.
140. Singh, S. (2017). "Optimize cloud computations using edge computing," in 2017 International Conference on Big Data, IoT and Data Science (BID), pp. 49–53, IEEE, 12.
141. Queralta, J. P., & Westerlund, T. (2021). Blockchain for Mobile Edge Computing: Consensus Mechanisms and Scalability, pp. 333–357. Springer International Publishing.
142. Qu, G., Wu, H., Li, R., & Jiao, P. (2021). Dmro: A deep meta reinforcement learning-based task offloading framework for edge-cloud computing. *IEEE Transactions on Network and Service Management, 18*(9), 3448–3459.